CHAPTER 5
RESULTS

This section reports the measurements that validate the design of Section 3, following the experiment matrix of Table 4.1 (Section 4.2). Each subsection gives the procedure specific to that experiment, the metric definitions, the results with sample size and uncertainty, an interpretation limited to what the data support, and the remaining gaps. Hardware emulation was disabled for every run (Section 4.1), so all measured figures are from the assembled robot.
Evidence classes. Every number in this section belongs to one of eight classes, and the class is stated wherever it is not obvious from the table caption:

- MEASURED: directly observed in an experiment on this robot and stored in `tools/eval/results/` or in the robot's own logs under `var/log/`.
- CALCULATED: derived from measured values or from documented design parameters by a stated formula.
- PUBLISHED REFERENCE: taken from a verified external source and never presented as a measurement of this robot.
- ESTIMATE: an explicitly qualified approximation resting on stated assumptions.
- PREDICTED: produced by a documented model and not an experimental observation.
- SOFTWARE-TEST: the result of a passing automated unit test exercising software logic, in most cases against a hardware mock; it establishes that the code behaves as documented for the inputs tested and is not a measurement of the physical robot.
- SIMULATED: the output of a software model with no physical measurement behind it; never presented as a measurement of real Wi-Fi behaviour, physical stopping time, or any other hardware quantity.
- [PENDING MEASUREMENT]: the required evidence does not exist; the protocol and an empty table are given instead.

Uncertainty. Means of independent trials carry a two-sided 95% Student t confidence interval (N - 1 degrees of freedom); medians carry a 95% percentile-bootstrap interval (10 000 resamples, fixed seed); success proportions carry a 95% Wilson score interval [**?**], which remains informative when every trial succeeds; and the 1 Hz resource series, whose samples are autocorrelated, carry a 95% interval from ten non-overlapping batch means rather than a

naive interval that would treat each second as independent. No hypothesis test is reported, and no difference between conditions is described as statistically significant. All of these figures are reproduced by `tools/eval/section5_stats.py`, whose output is stored in `tools/eval/results/section5_stats.md`; the per-CSV descriptive statistics are those of `tools/eval/results/summary.md`.

Data audit. Before analysis, every results CSV was cross-checked against the independent machine logs that the robot keeps regardless of the experiment (`var/log/p1_events.log`, which records every command sent to the Arduino and every line received from it with a millisecond timestamp, and `var/log/telemetry.log`, which stores the telemetry snapshot at about 1 Hz). Table 5.0 summarises the outcome. Rows written by the automated tools (E1, E7, E8) are machine-generated. Rows entered by hand through `record.py` (E3, E10, E11, E12) carry the time of data entry rather than the time of measurement: in each of these files successive trials are stamped about one second apart, which is the pace of typing, not of running the trials, so these sheets were transcribed after the session. Transcription is acceptable practice only when the robot's own logs agree with the transcribed values, and that agreement is what the last column of Table 5.0 records. Two results that the logs contradict or cannot support (E10 and E3) are retained with their tables but are marked [VERIFICATION REQUIRED] and are not used to support any claim. Because the robot's logs rotate (telemetry daily, seven kept), the log windows used here are preserved verbatim in `tools/eval/results/log_excerpts/`, and `tools/eval/section5_stats.py` reads them from there. The earlier text of this section is preserved verbatim in `research paper/archive/section5_results_pre-audit_2026-10-22.md`.

Table 5.0. Evidence status of every Section 5 experiment.

#	Experiment	Status	Source	Independent corroboration
E1	Control-plane latency	MEASURED	`latency.csv`, 420 rows, all `ok = 1`	Probe connections and dead-man events present in `p1_events.log` (2026-10-21)
E2	True control latency (240 fps)	[PENDING MEASUREMENT]	-	-
E3	Emergency-stop distance	MEASURED (transcribed); [VERIFICATION REQUIRED]	`estop.csv`, 23 rows	PWM 120 and 180 drive commands present in `p1_events.log`; no PWM 60 command on the trial date (2026-10-21; the only later PWM 60 commands belong to the automated 2026-10-22 sessions); distances inconsistent with the drive kinematics (Section 5.1)

#	Experiment	Status	Source	Independent corroboration
E4	Dead-man wheel-stop time (240 fps)	[PENDING MEASUREMENT]	-	Firmware-side stop interval measured from `p1_events.log` (Table 5.2b)
E5	Video and audio streaming	[PENDING MEASUREMENT]	-	-
E6	Communication range	MEASURED, partial (16 stations, 5 to 250 m, line of sight, no obstacle condition; no failure point reached, so usable range is at least 250 m by operator judgement only and its upper bound is [PENDING MEASUREMENT]; ping loss of 0 to 16% did not track distance)	`network.csv`, 16 rows	Directly logged by `netsweep.py` (ping + local RSSI), not hand-entered; distances and video/control judgements are operator-entered
E7	Fault injection, automated	MEASURED	`faults.csv`, 30 rows	10 P1 serial re-opens logged during the F3 campaign; P2 and P3 logs for that date have rotated out
E7	Fault injection, manual	MEASURED (F6, F7); F9 invalid	`fault_watch.csv`, 94 rows	Serial failures and dead-man trips logged in `p1_events.log`
E8	Resource utilisation	MEASURED	`resources.csv`, 3608 rows	Machine-logged; one condition contains two overlapping logger runs (Table 5.10a)
E9	GPS static precision and track	MEASURED, resolution-limited	`gps_static.csv` (1 summary row); `gps_track_live_2026-10-22.geojson`	Recomputed exactly from `telemetry.log` (Table 5.13a); track read from P1
E10	Ultrasonic range	[VERIFICATION REQUIRED]	`ultrasonic.csv`, 50 rows	Contradicted by `telemetry.log` (Section 5.6)
E11	Temperature and humidity (DHT11)	MEASURED (transcribed) for sensor values; [VERIFICATION REQUIRED] for the error figures	`dht11_environmental.csv`, 5 rows	Sensor values agree with `telemetry.log`; `error_c` is exactly 0.0 on every trial (Section 5.6.3)
E12	Gas sensor response	MEASURED (transcribed)	`gas.csv`, 5 rows	Peaks and baselines visible in `telemetry.log`

#	Experiment	Status	Source	Independent corroboration
E13	Field trial	[PENDING MEASUREMENT]	No `field.csv`	No per-run record exists (Section 5.7)
E14	Operator usability (SUS)	[PENDING MEASUREMENT]	No `sus.csv`	-
E15	Cost	RECEIPT-BACKED (BDT; USD at indicative rate)	Table 5.19	Authors' itemised receipts, 37,158 BDT; a few parts not itemised

5.1 Control Latency and Emergency-Stop Response

Procedure. The control-plane latency probe `tools/eval/wslatency.py` (E1, Table 4.1) was run on the Pi itself and connected to P1 over the loopback interface (`127.0.0.1`, as recorded in P1's connection log), so its figures describe the robot-internal path (WebSocket server, P1, USB serial link, firmware and telemetry return) and exclude the Wi-Fi link and the operator laptop. Two jittered sessions were recorded, on 2026-10-07 (label `on-pi`) and on 2026-10-21 (label `on-pi-2026-10-21`), each with 30 valid trials per metric and no discarded trial (all 420 rows carry `ok = 1`). A third session with the send-phase jitter disabled (`on-pi-phaselocked`) was recorded to show the bias the jitter removes (Section 4.2).
Metrics. `hello_rtt` is the authentication request-response round trip and is the only metric that does not wait on telemetry. `motor_start` (drive command to `fw_state` ACTIVE), `motor_stop` (stop command to `fw_state` READY) and `pan_echo` (pan command to the new angle in telemetry) are telemetry-confirmed: because the firmware emits telemetry every 500 ms and P1 rebroadcasts on a 200 ms timer, each is an upper bound on the true command latency that can exceed it by up to 700 ms and by about 350 ms on average (Section 3.6.2). `deadman_trip` is the interval from the probe's last command to `fw_state` reporting the stopped state, again telemetry-confirmed. `deadman_panic`, recorded only in the 2026-10-21 session, is the interval from the same last command to the firmware's PANIC line as time-stamped in P1's event log, which is not subject to the telemetry quantisation. All values are in ms.

Table 5.1. Control-plane latency, jittered sender, both sessions, `tools/eval/results/latency.csv`. MEASURED. All values in ms; 95% CI of the mean (Student t).

Metric	Session	N	Mean	95% CI	SD	Min	Max	Median	p95
hello_rtt	2026-10-07	30	46.8	37.7 to 56.0	24.6	19.6	116.9	41.0	75.8
hello_rtt	2026-10-21	30	138.0	115.7 to 160.3	59.7	30.1	255.9	115.9	247.7
motor_start	2026-10-07	30	328.2	277.4 to 379.0	136.1	156.8	761.1	306.3	573.5
motor_start	2026-10-21	30	275.4	251.9 to 298.9	62.9	180.1	434.1	264.3	383.3

Metric	Session	N	Mean	95% CI	SD	Min	Max	Median	p95
motor_stop	2026-10-07	30	314.3	284.5 to 344.2	80.0	207.9	606.7	300.0	393.9
motor_stop	2026-10-21	30	302.9	281.4 to 324.4	57.6	195.5	385.6	312.2	381.4
pan_echo	2026-10-07	30	387.7	365.2 to 410.3	60.4	285.9	495.0	381.1	480.8
pan_echo	2026-10-21	30	389.6	359.8 to 419.4	79.7	265.6	637.1	378.2	481.7
deadman_trip	2026-10-07	30	2409.1	2366.2 to 2452.1	115.1	2200.9	2598.4	2409.7	2584.1

The p95 column is the linear-interpolation 95th percentile of 30 observations; at this sample size it rests on the two largest values and is reported as a descriptive figure, not as a stable tail estimate.

Table 5.2. Phase-locked sender (label `on-pi-phaselocked`, 2026-10-07), same path, showing the biased distribution the jitter removes. MEASURED. All values in ms.

Metric	N	Mean	SD	Min	Max	Median	95% CI of median	p95
motor_start	30	407.1	62.8	266.2	504.9	393.9	392.0 to 398.4	496.0
motor_stop	30	403.3	19.9	392.6	507.2	400.7	399.2 to 401.5	405.7
pan_echo	30	492.1	82.9	150.3	609.8	500.3	498.4 to 501.6	607.0

Table 5.2a. Dead-man trip paired with the PANIC notice, 2026-10-21 session (label `on-pi-2026-10-21`), 30 trials at drive speed 150. MEASURED. All values in ms.

Metric	Timed from	N	Mean	95% CI	SD	Min	Max	Median	p95
deadman_trip	telemetry `fw_state`	30	2435.9	2394.5 to 2477.3	110.8	2197.5	2593.2	2467.5	2582.8
deadman_panic	P1 event-log PANIC line	30	2079.1	2068.7 to 2089.6	27.9	2022.3	2122.3	2076.5	2120.8

`deadman_panic` is timed from the probe's WebSocket send, so it still contains the WebSocket and P1 processing before the command reaches the serial port. P1's event log removes that leg: it time-stamps every command written to the Arduino (`DEBUG_TX`), the firmware's acknowledgement of it (`ACK`), and the `PANIC DEADMAN` line, all on one clock. Table 5.2b decomposes the 30 trials of Table 5.2a using those entries; every probe trial matched exactly one PANIC line.

Table 5.2b. Dead-man interval decomposed on P1's clock, 2026-10-21, `var/log/p1_events.log` matched to Table 5.2a. CALCULATED from MEASURED log timestamps. All values in ms.

Interval	N	Mean	95% CI	SD	Min	Median	Max
P1 sends last drive command → P1 receives PANIC line	30	2022.5	2019.7 to 2025.3	7.5	2009.0	2024.0	2042.0
P1 receives ACK of that command → P1 receives PANIC line	30	2002.0	1998.7 to 2005.2	8.7	1986.0	2002.5	2022.0
P1 sends last drive command → P1 receives its ACK	30	20.5	18.8 to 22.2	4.6	6.0	21.0	29.0
Probe `deadman_panic` minus first row (WebSocket and P1 leg)	30	56.6	46.8 to 66.4	26.2	6.3	52.9	101.3

Analysis. The dead-man result is the strongest timing evidence in the paper. In the firmware, `taskSafety()` calls `emergencyStop()` and then `sendPanic()` in the same pass (`arduino/arduino.ino`, lines 79 to 86), so the PANIC line marks the instant at which drive is removed electrically. Measured on P1's clock from the moment the last command left P1 to the moment the PANIC line arrived back, the interval never exceeded 2042 ms in 30 of 30 trials (mean 2022.5 ms, 95% CI 2019.7 to 2025.3 ms), and this interval still includes the serial transit in both directions. It therefore lies below the analytical stop bound of approximately 2060 ms of Section 3.5.2 (2000 ms window + 10 ms supervisor period + about 50 ms longest loop pass) in every trial, by at least 18 ms. The interval from the firmware's acknowledgement to its PANIC line, in which the return leg cancels because both lines travel the same path, has a mean of 2002.0 ms (95% CI 1998.7 to 2005.2 ms), which agrees with the 2000 ms dead-man window; its spread (min 1986, max 2022 ms) shows that P1's receive timestamps carry a jitter of the order of 15 ms from USB-serial buffering and event-loop scheduling, which is the resolution of this method. The probe's `deadman_panic` figure of 2079.1 ms exceeds the bound only because it also contains the WebSocket and P1 leg before the command reached the Arduino (mean 56.6 ms, last row of Table 5.2b); the earlier draft's statement that this figure "sits directly on" the bound conflated the two. The difference between the paired `deadman_trip` and `deadman_panic` means, 356.8 ms, is consistent with the mean telemetry-reporting delay of about 350 ms predicted in Section 3.6.2, and the drop in SD from 110.8 ms to 27.9 ms shows that most of the spread in the telemetry-confirmed figure belongs to the reporting path. What these log measurements establish is the time at which the firmware removed drive. The time at which the wheels physically come to rest after that, which includes mechanical run-down, is the subject of E4 and is not yet measured.

The command-confirmation metrics (`motor_start`, `motor_stop`, `pan_echo`) have means

between 275 and 390 ms in both sessions and all fall inside the 700 ms confirmation bound, consistent with the upper-bound model of Section 3.6.2; they are upper bounds, and the true command latency is lower by an unknown amount up to 700 ms per trial until E2 is measured. The two sessions agree closely for `motor_stop` and `pan_echo` (overlapping confidence intervals). They do not agree for `hello_rtt`: its mean rose from 46.8 ms (95% CI 37.7 to 56.0) on 2026-10-07 to 138.0 ms (115.7 to 160.3) on 2026-10-21, with non-overlapping intervals. Because both sessions ran over loopback, this difference is not a network effect; it reflects a change in WebSocket and authentication processing time on the Pi between the two days, whose cause (background load, software state) was not recorded. Consequently `hello_rtt` is not a stable floor for the control path, and the earlier description of it as bounding "the pure network-and-WebSocket contribution" is withdrawn: no Wi-Fi figure exists for E1. The phase-locked run (Table 5.2) illustrates the artifact the jitter is designed to remove: its medians collapse onto the rebroadcast grid (motor_stop median 400.7 ms, bootstrap interval 399.2 to 401.5 ms, SD 19.9 ms against 80.0 ms jittered), a deceptively tight distribution with a shifted mean, so the jittered figures of Table 5.1 are the primary result, as recorded in `tools/eval/README.md` and discussed in Section 6.4. As recorded in `tools/eval/wslatency.py`, the 0 to 200 ms jitter cannot fully decorrelate a trial from the 500 ms firmware telemetry cycle, so the jittered spread still understates the full quantisation spread.

Comparison with the literature. Human-factors studies of teleoperation identify command and video latency, frame rate and field of view as the main system factors that degrade operator performance [**?**]. The figures here are robot-internal upper bounds measured over loopback and cannot be set against those human-performance results until the end-to-end latency over Wi-Fi (E2) is measured.

The true control latency (E2) and the physical dead-man wheel-stop time (E4) require a 240 fps camera and have not been collected; the emergency-stop stopping distance (E3) is reported below.

[PENDING MEASUREMENT: E2 true control latency.] Protocol: run the dashboard on the operator laptop over the Wi-Fi cell of Section 3.2, film the operator screen and the drive wheels together at 240 fps (4.17 ms per frame) with `tools/eval/record.py latency_video`, count frames from key press to first wheel movement, and record 30 trials. Metric: key-press-to-first-wheel-motion latency in ms, reported as mean with 95% CI, median and maximum, with a resolution of one frame. This metric gives the true end-to-end control latency that the robot-internal, telemetry-confirmed figures of Table 5.1 cannot.

Table 5.3. True control latency (E2), `latency_video.csv`. [PENDING MEASUREMENT]

trial	label	fps	frames	latency_ms

[PENDING MEASUREMENT: E2 analysis to be filled when latency_video.csv data is collected; the result will be reported as the true control latency and compared with the telemetry-confirmed upper bounds of Table 5.1.]

Emergency-stop stopping distance (E3). The robot was driven forward on a level concrete floor at three commanded speeds (PWM 60, 120, 180); on each trial the operator pressed the dashboard emergency-stop button as a reference mark on the robot crossed a fixed line, and the distance from that line to the mark at rest was tape-measured and later entered with `tools/eval/record.py estop` (entry timestamps 2026-10-21). The emergency-stop button issues the ramped stop S of Section 3.5.3 (Table 7). Stop times (`stop_time_s`) were not recorded.

Table 5.4. Emergency-stop stopping distance (E3), concrete surface, `tools/eval/results/estop.csv`. MEASURED (operator-recorded, transcribed); [VERIFICATION REQUIRED] for all three speeds, since the kinematic check below applies to every row and the PWM 60 row also lacks any logged drive command. Distance in cm.

Speed (PWM)	N	Mean	95% CI	SD	Min	Max	Median	95% CI of median
60	7	42.7	36.4 to 49.0	6.8	34	54	41	38 to 48
120	9	75.9	61.5 to 90.3	18.7	34	92	78	60 to 90
180	7	91.3	85.1 to 97.4	6.7	78	98	92	90 to 97

Analysis and verification. Two independent checks place this table outside the evidence used to support any claim. First, P1 logs every drive command it sends to the Arduino, and P1 forwards the requested speed unchanged apart from clamping to the 180 cap (`pi/p1_control/safety.py`). On 2026-10-21 the log contains 12 `F180` commands (11:45 to 11:54 UTC) and 27 `F120` commands, which could correspond to the PWM 180 and PWM 120 trials, but it contains no `F60` command on that day. The only `F60` commands in the retained logs (2026-09-25 to 2026-10-23) are 44 on 2026-10-22, between 21:16 and 21:41 UTC, which fall in the automated `bench_motor.py` and `estop_auto.py` sessions (Table 5.4 text below) and not in the manual trials of Table 5.4, so the seven PWM 60 trials of Table 5.4

still have no corresponding drive command. Second, the distances are inconsistent with the drive kinematics by more than an order of magnitude (CALCULATED). With the 60 RPM no-load motors and 65 mm wheels of Table 2, the no-load ground speed at the motors' rated 12 V, which the PWM 180 cap is set to approximate (`pi/common/protocol.py`), is (60/60) x pi x 0.065 m = 0.20 m/s. The 120 ms ramp bound is not an assumption carried over from Section 3.5.2 unchecked: it was re-derived directly from the firmware source for this audit. `arduino/config.h` sets `MOTOR_RAMP_STEP` to 15 and `MOTOR_SERVICE_PERIOD_MS` to 10, and `Motors::rampToward` (`arduino/motors.cpp`) moves the current PWM toward the target by one `MOTOR_RAMP_STEP` on every service tick, so a stop from the PWM 180 cap takes ceil(180/15) = 12 ticks x 10 ms = 120 ms regardless of which stop path in Table 7 issued it, since the emergency-stop button issues the same ramped S as a key release. At that speed the ramp covers at most 2.5 cm, and even the telemetry-confirmed `motor_stop` upper bound of about 0.3 s adds at most about 6 cm; covering the measured 78 to 98 cm at PWM 180 would instead require 3.8 to 4.8 s of travel at full no-load speed, and the 34 to 54 cm at PWM 60 would require still longer at one third of the voltage. The recorded distances therefore cannot be the travel after the stop command under the documented stop design and motor specification.

This leaves three distinct questions, which this audit answers separately rather than folding into one verdict. (i) Is the kinematic *model* itself correct, independent of any measurement? Yes: every constant it uses (the 15-PWM ramp step, the 10 ms service period, the 60 RPM no-load rating and 65 mm wheel diameter of Table 2, and the 180 PWM cap) was re-read from the firmware source and from Table 2 for this check, not assumed, so the 0.20 m/s no-load speed and the 120 ms, 2.5 cm stopping-distance bound are CALCULATED and reproducible by any reader from the cited files; this holds at every PWM setting the firmware accepts, including the three tested here, because the ramp step and service period do not depend on the starting PWM. (ii) Does the model agree with the E3 measurements? No: the measured distances exceed the model's bound by one to two orders of magnitude at all three speeds, so the model is CONTRADICTED by Table 5.4, not confirmed by it. (iii) Has the physical stop been validated by filming the wheels, independent of the model? No: that is E4 (Table 5.5), which remains [PENDING MEASUREMENT]; nothing in this analysis substitutes for it. Possible explanations for the gap between (i) and (ii) are a difference between the procedure as run and as described (for example, the distance including the operator's reaction before the press), a motor or wheel specification that differs from Table 2, or a recording error; none can be resolved from the stored data. The earlier interpretation of the speed dependence as the signature of a bounded-time ramp stop is withdrawn, since that ramp predicts centimetre-scale distances. The table is retained because it is an operator record, but the claim that the operator-triggered stop brings the robot to rest over a short, speed-bounded distance remains untested. The sample is also below the 20-trial-per-speed target of Table 4.1.

[PENDING MEASUREMENT: E3 repeat.] Protocol: repeat with `tools/eval/estop_assist.py`, which sends the stop over the control WebSocket at the operator's key press so that

each trial appears in P1's log, film each trial at 240 fps to obtain `stop_time_s` and the speed at the line, record the drive speed actually reached, and enter distances during the session; 20 trials per speed.

A 2026-10-22 session verified, by log correlation, that the command/stop path itself functions correctly at all three PWM levels (`tools/eval/estop_auto.py`, `tools/eval/results/estop_auto.csv`, 15 trials, every row's `distance_cm` written as the literal `PENDING_MEASUREMENT` rather than estimated) but captured no physical distance, since that still requires an operator with a tape measure. A separate, explicitly labelled SIMULATED model (`research-analysis/sim/estop_distance_sim.py`) gives the size of the gap above a concrete number instead of a prose estimate: combining the same verified kinematic bound used above with the already-measured Table 5.1 `motor_stop` latency distribution as a generous stand-in for stop-command-to-halt delay still predicts only 2.4 to 8.7 cm across the three speeds, 10 to 18 times smaller than Table 5.4's tape-measured means; full method and figures in `research-analysis/e3_evaluation_2026-10-22.md`. E3 remains incomplete and `[VERIFICATION REQUIRED]`.

[PENDING MEASUREMENT: E4 dead-man physical stop time.] Protocol: with `tools/eval/record.py deadman_physical`, cut the command stream and film the wheels at 240 fps, timing from the last heartbeat to wheels stationary; record whether the PANIC DEADMAN line appeared; 20 trials. This is the direct test of the physical stop; Table 5.2b already establishes from the firmware side that drive is removed within 2042 ms of the last command in every logged trial.

Table 5.5. Dead-man wheel-stop time (E4), `deadman_physical.csv`. [PENDING MEASUREMENT]

trial	speed_pwm	stop_s	panic_seen

[PENDING MEASUREMENT: E4 analysis to be filled when deadman_physical.csv data is collected; the filmed wheel-stop time will be compared with the approximately 2060 ms bound of Section 3.5.2 and with the firmware-side interval of Table 5.2b.]

Limitations. No E1 figure includes the Wi-Fi link. The latency metrics other than `hello_rtt` and the log-timed dead-man intervals are upper bounds. The log-timed method resolves the firmware stop to about 15 ms and does not observe the wheels. E3 is unverified.

5.1a Software Verification: Automated Unit Tests

The timing guarantees analysed in Section 3.5.2 and measured in Table 5.2b rest on firmware-facing and server-side logic whose correctness is checked separately by automated unit tests against `pi/common/mock_hardware.py`'s `MockArduino` model and the Pi-side validation code, under `MOCK_HARDWARE` semantics. This is SOFTWARE-TEST evidence: it establishes that the code behaves as documented at the inputs tested, and it does not operate the physical Arduino firmware, the motors, or the radio link.

`docs/TEST_REPORT.md` reports 116 passed, 0 failed, run on a Raspberry Pi 4 on 2026-10-08; that figure is quoted here, not rerun, and `docs/TEST_REPORT.md` is unchanged by this revision. Five further tests in `pi/tests/test_ice_config.py` (STUN-optional ICE configuration, no TURN relay in local mode under two parametrisations, and per-client ICE rate limiting) were committed about 1 h 40 min after the 116-count, the same morning, and were never folded into that report's test-file table; the 116-test figure is correct for what existed when it was recorded and is kept dated and intact rather than silently combined with a later count.

A fresh rerun of the full existing suite on 2026-10-22, in the same Python environment the live `robot-watchdog.service` uses, passed 121 of 121 tests with 0 failures: the 116 historical tests plus the five `test_ice_config.py` tests. This is a separately dated, independently executed result reported alongside, not in place of, the historical figure.

The key lockout of Section 3.8 (5 failures within 300 s, host refused for 300 s even with the right key) is covered only at this level. `pi/tests/test_controller_access.py`, which is part of the 121-test suite and configures the gate with `max_failures=5` and `lockout_s=300`, passed 25 of 25 tests when run alone on 2026-10-23, including the lockout, its refusal of a correct key and the expiry of old failures. No lockout trial was run on the robot. The 3000 ms operator-side telemetry timeout of Section 3.7.3 was not measured in any experiment and is a specification value only.

A further set of boundary-condition tests, written for this verification pass and kept outside `pi/tests/` under `research-analysis/tests/` (not merged into the count above), passed 25 of 25 when rerun on 2026-10-23: 17 on the dead-man boundary, the command validator and the command framing, and 8 that exercise the corrected mock behaviour in isolation (`test_deadman_patch_isolated.py`). The 17-test group first passed on 2026-10-22; its dead-man arming and boundary assertions were revised after the native firmware test described below (Table 5.5d), and the counts here are for the revised tests. They target exact boundary values that the existing 121 tests do not probe: the dead-man timer's alive/tripped boundary at exactly `DEADMAN_MS` (2000 ms) and one ms either side of it, and the re-arming of the timer by pan, tilt and status-request commands, as the firmware does; the server-side command validator's PWM (0-180) and servo-angle (0-180) clamps at their exact limits and its sequence-number monotonicity check at the exact boundary; and the command-framing length cutoff (`MAX_COMMAND_CHARS = 8`) at exactly 8 and 9 characters.

Table 5.5a. Automated unit tests: historical record, fresh rerun and supplementary boundary tests. SOFTWARE-TEST.

Test set	Scope	Count	Result	Date
Historical suite	`pi/tests/` as it stood on 2026-10-08	116	116 passed, 0 failed	2026-10-08 (`docs/TEST_REPORT.md`, quoted, not rerun)
Fresh rerun, same suite	`pi/tests/`, full dependency environment, including the 5 `test_ice_config.py` tests added after the historical count	121	121 passed, 0 failed	2026-10-22
Supplementary tests	`research-analysis/tests/`: dead-man boundary, command-validator boundary, command-framing boundary (17), and the corrected mock in isolation (8)	25	25 passed, 0 failed	2026-10-23 (the 17-test group first passed on 2026-10-22)

These results establish that, for the P1-side `MockArduino` model as corrected on 2026-10-22 (addendum below), the dead-man alive/tripped boundary (trip at elapsed $\geq$ 2000 ms), the re-arming of the timer by pan, tilt and status commands, the PWM and servo-angle clamps, the sequence check and the command-length cutoff behave as documented in `pi/common/mock_hardware.py`, `pi/p1_control/safety.py` and `pi/common/protocol.py`, for the inputs exercised. **These 146 tests (121 + 25) characterise the P1-side `MockArduino` software model, not the Arduino firmware's own source**; none of them operates the physical Arduino firmware, the motors, or a real radio link, and none of them substitutes for the dead-man wheel-stop measurement of Table 5.5 (E4), which remains [PENDING MEASUREMENT]: 146 is a count of software-logic tests against mocks, not of trials on the assembled robot, and it does not establish measured motor-stop performance. The next paragraph tests the firmware source itself. It found that the mock, as first written, treated the boundary direction and the arming by pan, tilt and status commands differently from the firmware, and the mock was then corrected to match.

Native firmware verification (ACTUAL SOFTWARE TEST), 2026-10-22. The tests above exercise `MockArduino`, a from-scratch Python re-implementation; they say nothing about whether its modelled boundary and arming rules match `arduino/command_parser.cpp` and `arduino/robot_state.cpp` themselves. To close that gap without hardware, this revision compiles the real, unmodified firmware sources (`protocol.cpp`, `utils.cpp`, `robot_state.cpp`, `motors.cpp`, `servos.cpp`, `telemetry.cpp`, `command_parser.cpp`, all under `arduino/`, byte-identical to the files that are flashed to the board) with `g++` against a small host-side stub of the Arduino API (`research-analysis/native/fake_arduino/`: `millis()`, `pinMode`/`digitalWrite`/`digitalRead`/`analogWrite` recorded against fake pin state, and a `Serial` object with an injectable RX queue and a capturable TX string), producing a native test binary (`research-analysis/native/build/deadman_native_tests`) that drives the actual compiled command-parsing and dead-man logic. The one exception is `arduino.ino`'s `taskSafety()`, a `static` function in that translation unit that a separate test binary cannot link against directly; it is reproduced verbatim (same lines, same comments) in `research-analysis/native/tests/mirrored_safety_task.h`, cited against its exact source location, rather than re-derived. This is a software test, not hardware-in-the-loop testing: no board, no UART electrical timing, and no ATmega-specific PWM behaviour is exercised. Build and run output: `research-analysis/output/native_deadman_tests_2026-10-22.txt`. Result: 10 test functions, 27 checks, 10/10 passed.

Table 5.5d. Native test of the real, compiled firmware source (not the `MockArduino` model). ACTUAL SOFTWARE TEST, 2026-10-22.

Check	**Real-firmware result**	**Agrees with the `MockArduino`-based claim above?**
Alive one ms before the boundary (elapsed = 1999 ms)	alive, motors unchanged	Yes
Exactly at the boundary (elapsed = 2000 ms)	**tripped** (`taskSafety()`'s check is `>= DEADMAN_TIMEOUT_MS`, an inclusive upper bound)	**No** – the mock's <= reports "alive" at this exact instant; the firmware reports "tripped"
One ms past the boundary (elapsed = 2001 ms)	tripped, motors stopped	Yes
Motor EN line and all four PWM pins on trip	EN line LOW, all four BTS7960 PWM pins (`LEFT_RPWM`, `LEFT_LPWM`, `RIGHT_RPWM`, `RIGHT_LPWM`) 0	n/a – the mock has no GPIO pin model, so this safe-state property was previously untested at any level

Check	Real-firmware result	Agrees with the `MockArduino`-based claim above?
Repeated pan (`P90`) commands while a drive command is left to expire	**re-arms the dead-man** (firmware fault stays `NONE`); `command_parser.cpp`'s own comment reads "Any well-formed command counts as comms activity for the dead-man timer, even a query or heartbeat"	**No** – `pi/common/protocol.py`'s `DEADMAN_ARMING_COMMANDS` deliberately excludes pan/tilt/status, and the mock and the Table 5.5a tests built on it report no re-arm
Tilt (`T45`) and status (`?`) commands, same setup	**re-arms the dead-man**, same mechanism	**No**, same reason
First command sent immediately after a trip	**rejected** (`NACK F REJECTED`): the state-machine gate (`checkState()`) still sees `MODE_PANIC` for that one command, because `handleLine()` updates `lastCommandTime` *before* checking state; the very next 10 ms safety pass then sees fresh activity and auto-clears the fault (`EVT DEADMAN_CLEARED`), so a second attempt succeeds	n/a – `MockArduino` has no `MODE_PANIC`/`MODE_ESTOP` gate at all (by its own docstring) and accepts the first post-trip command immediately with no rejection, confirmed by re-running it directly: `research-analysis/output/native_deadman_tests_2026-10-22.txt` records both behaviours side by side
Unknown opcode, out-of-range speed	NACKed, motors unaffected	Yes
Heartbeat (`H`) before the boundary	re-arms, motors unchanged	Yes

What this changes. Two behaviours of the original `MockArduino` model did not match the real firmware, although the mock-based tests had treated both as documented: the boundary direction (trip is at `>= 2000` ms, not `> 2000` ms) and the pan/tilt/status non-arming rule (the real firmware re-arms on *any* valid command; only the mock treats pan/tilt/status specially). Both are one-line, well-commented design choices in `arduino/command_parser.cpp` and `arduino/arduino.ino`, not bugs introduced by this check. The recovery mechanism – reject once, auto-clear within one 10 ms safety tick, succeed on the next attempt – is new evidence either way: it was not covered by any existing test (mock-based or native) before this revision, and it shows the dead-man's recovery is automatic and fast (bounded by the 10 ms safety-task period) rather than requiring an operator-visible retry loop. The motor EN-line and PWM safe-state check is likewise new: no previous test, mock or native, inspected pin-level state at all. None of this is hardware-in-the-loop evidence; the ATmega's actual PWM duty cycle, the EN line's real voltage, and UART timing on physical silicon remain [PENDING MEASUREMENT] along with E4.

Addendum, 2026-10-22 (later same day): mock corrected to match the firmware. `pi/common/mock_hardware.py`'s `_check_deadman` and `pi/common/protocol.py`'s `DEADMAN_ARMING_COMMANDS` have since been patched (two one-line behavioural changes, reviewed as a diff before being applied; proposal and verification preserved in `research-analysis/proposed_patch/`) so that `MockArduino` now trips at the same inclusive `elapsed >= DEADMAN_MS` boundary as the firmware, and now re-arms on pan, tilt and status commands exactly as `command_parser.cpp` does. The "No" answers in the right-hand column of Table 5.5d describe the *pre-patch* mock and are left unchanged as the historical record of what the native test found; as of this addendum they no longer describe the current `pi/common/mock_hardware.py`. The patch was verified against the full protected `pi/tests/` suite (121/121 still pass; none of those historical tests asserted the specific boundary-equality or pan/tilt/status-exclusion behaviour that changed) and against updated boundary tests in `research-analysis/tests/test_deadman_boundaries.py` (6/6 pass, three of them renamed and their expected outcomes flipped to match the corrected behaviour). This changes only the fidelity of the P1-side software model to the firmware; it does not change the firmware itself, and it does not supply any new physical measurement – E4 (dead-man wheel-stop) remains [PENDING MEASUREMENT].

5.1b Dead-Man Heartbeat-Loss Robustness (SIMULATED)

All figures in this subsection are SIMULATED: they come from a software model (`research-analysis/sim/deadman_robustness_sim.py`) built from two literal, verified source constants, not from any measurement of the robot's Wi-Fi link. They do not estimate the real link's heartbeat loss rate: only ICMP ping loss to the access point exists (0 to 16% per 100-ping burst, E6, Section 5.3), which is a different quantity, and they do not model physical stopping distance.

Table 5.2b measures how long the dead-man takes to trip once commands stop arriving, on a link that was not itself dropping packets during the trial. It does not by itself show how many consecutive lost heartbeats the 2000 ms dead-man window (`DEADMAN_MS`, `pi/common/protocol.py:67`) can tolerate at the dashboard's configured 500 ms heartbeat period (`HEARTBEAT_PERIOD_MS`, `dashboard/src/lib/protocol.ts:11`), nor how mission-level trip probability scales with an assumed per-heartbeat loss rate. This subsection reports that as a software model: a deterministic boundary analysis for the tolerated-loss margin, and a Monte Carlo sweep (300 s simulated missions, 50 000 trials per assumed loss probability, fixed seed 20261022) over an assumed per-heartbeat loss probability.

At the configured 500 ms heartbeat period and the firmware's inclusive trip condition (`elapsed >= DEADMAN_TIMEOUT_MS`, Table 5.5d), 2 consecutive lost heartbeats leave a 1500 ms gap and cause no trip. A 3rd in a row leaves 2000 ms of silence at the moment the 4th heartbeat is due, and the firmware trips at that instant, before the 4th can arrive (idealised, with no transit

jitter).

Table 5.5b. Consecutive-loss margin by heartbeat period. SIMULATED; CALCULATED from `DEADMAN_MS = 2000` ms, the firmware's >= trip condition and the heartbeat period, with no transit jitter. A loss count is tolerated if the next heartbeat can still arrive before 2000 ms of silence.

Heartbeat period (ms)	Consecutive losses tolerated	Losses that guarantee a trip
200	8	9
500 (configured system)	2	3
1000	0	1
2000	none	0 (trips with no loss)

Table 5.5c. Mission-level trip probability vs. an assumed per-heartbeat loss probability. SIMULATED. 300 s missions, 50 000 trials per value, seed 20261022.

Assumed per-heartbeat loss probability	Trip rate	95% CI half-width
0.01	0.05%	+/- 0.02 pp
0.02	0.47%	+/- 0.06 pp
0.05	6.84%	+/- 0.22 pp
0.10	42.04%	+/- 0.43 pp
0.15	82.34%	+/- 0.33 pp
0.20	97.94%	+/- 0.12 pp
0.30	100.00%	+/- 0.01 pp
0.50	100.00%	+/- 0.00 pp

Reading this result. The curve is steep because a trip requires 3 independent consecutive losses, so the trip probability scales roughly with the cube of the per-heartbeat loss probability at small values: an assumed 5% per-heartbeat loss already produces a simulated trip probability of about 7% over 5 minutes, 10% produces 42%, and by 15% a trip is far more likely than not (82%). This is a direct consequence of the chosen 500 ms/2000 ms ratio, not evidence of a flaw, and it shows that a longer effective heartbeat period (for example from added processing delay, or a throttled dashboard timer) would shrink the tolerated-loss margin quickly (Table 5.5b); at 1000 ms a single lost heartbeat trips.

Limitations. The model assumes the configured 500 ms heartbeat period, but the logged period was 1.00 s in the minutes before the F7 injection and before the unplanned stalls (Section 5.4, Table 5.9d); at 1 s, Table 5.5b gives no tolerated loss, so Table 5.5c understates the trip probability for those sessions. The per-heartbeat loss probability swept here is an assumed input, not fitted to any measurement; it exists to show the shape of the sensitivity curve, not to claim the real link's loss rate. Loss is modelled as independent per heartbeat with no burst correlation; real Wi-Fi loss is typically bursty, which would trip the dead-man sooner at the same mean loss rate than this model predicts, so the model's trip probabilities at a given mean loss rate are a lower bound relative to a burst-loss model. No jitter or transit delay is modelled, only binary loss per heartbeat slot: the third consecutive loss trips only in this zero-jitter idealisation, because the next heartbeat is due exactly at 2000 ms, and with real arrival jitter the outcome at that point is decided by timing, so the two-loss tolerance of Table 5.5b is the margin that holds without the idealisation. A single 300 s mission length was swept. Tables 5.5b and 5.5c use the firmware's inclusive boundary confirmed by the native test (Table 5.5d). This subsection is not a measurement of the robot's actual wireless packet loss, physical stopping time, or real-world communication reliability; Section 5.3 (communication range) reports only a partial sweep of ICMP ping loss, not heartbeat loss.

5.2 Video and Audio Streaming Performance

This subsection reports the delay of the two-way media path (E5, Table 4.1). No valid streaming measurement exists; the experiment is entirely pending.

Procedure. Glass-to-glass delay is measured with `tools/eval/record.py glass2glass` by placing a millisecond clock in the robot camera's view, photographing the clock and its copy rendered in the operator video panel in the same frame, and differencing the two readings. This is a simpler and coarser procedure than the electronic glass-to-glass measurement system of Bachhuber and Steinbach (0.5 ms precision) [**?**], whose metric it reports; its resolution is limited by the displayed clock and the camera exposure. The audio delay is measured in each direction by the same principle, recording an impulse (a clap) and its reproduction at the far end on one recorder. Conditions: one viewer (`viewers-1`), two viewers (`viewers-2`, to expose the per-session encoders of Section 3.9.1), and the operator at 20 m (`wifi-20m`); 20 trials per condition.

Metrics. Video glass-to-glass delay (ms): displayed-clock reading subtracted from the real-clock reading in one photograph. Audio mouth-to-ear delay (ms), robot to operator and operator to robot. Received video frame rate (frames/s) and bitrate (kbit/s), read from the operator browser's WebRTC statistics over each trial. Packet loss (%), from the same statistics. Each is reported as mean with 95% CI, median and maximum.

Table 5.6. Glass-to-glass video delay (E5), `glass2glass.csv`. [PENDING MEASUREMENT]

trial	label	real_ms	robot_ms	delay_ms

Table 5.6a. Media summary per condition (E5). [PENDING MEASUREMENT]

Condition	N	Video delay mean	Audio R→O	Audio O→R	Frame rate	Bitrate	Loss
viewers-1							
viewers-2							
wifi-20m							

What existing data can and cannot show. The resource log of E8 records the Wi-Fi interface transmit rate (`tx_kbps`), but it counts all traffic on the interface. In every driving condition the Pi desktop was also being viewed remotely (`display_mode` = `vnc` on all samples), and the median transmit rate without an operator media stream (`driving`, 1152 kbit/s) was no lower than with one (`driving+media`, 1127 kbit/s). The interface counter therefore cannot isolate the media bitrate, and no bandwidth, frame-rate, delay or audio-quality figure is reported for this robot. The design parameters remain the specification against which E5 will be judged: 640x480 at 10 fps with a 500 kbit/s target for video, and 32 kbit/s Opus [**?**] in 60 ms packets for audio (Section 3.9, Table 1a).

[PENDING MEASUREMENT: E5 analysis to be filled when glass2glass.csv and the WebRTC statistics are collected; the delays will be interpreted against the parameters above and the two-way victim-interaction claim (RQ4) assessed.]

Limitations. Until E5 is measured, the paper makes no quantitative claim about media delay or quality.

5.3 Communication Range and Link Degradation

This subsection reports the Wi-Fi range sweep (E6, Table 4.1). The sweep is partial. Sixteen line-of-sight stations were logged in two sessions, 5 to 250 m from the access point (`tools/eval/results/network.csv`): N1 to N6 on 2026-10-22, and N7 to N10 and R1 to R6 on 2026-10-23. No obstacle condition was run, and no station at which the operator judged video or control unusable was reached, so no upper bound on usable range exists. The principal finding is negative and methodological: ICMP packet loss to the access point ranged

from 0 to 16 percent per 100-ping burst without tracking distance, while the operator judged video and control usable at every judged station.

Procedure. In the outdoor open area, at line-of-sight stations moving outward from the access point, `tools/eval/netsweep.py --target <router-ip> --label <Nk> --distance-m <d> --obstacles <desc> --video <good|choppy|lost> --control <yes|degraded|no>` was run on the Pi. It sends 100 ICMP echo requests at 0.2 s intervals to the router (192.168.0.1), so the measured path is the Pi-to-router radio link, and it reads the signal level at the start and end of the burst, the negotiated transmit bitrate, the channel, and logs loss and round-trip time. The operator supplies the two usability judgements on the command line at the station. Stations N1 to N6 were run without `--video`/`--control`; N7 to N10 were run with them. R1 to R6 are a deliberate re-run, at the same six distances as N1 to N6, with the judgements recorded (`notes` = `rerun`), added because the original six rows lacked them; the N1 to N6 rows are retained unchanged.

Metrics. Packet loss (% of pings lost), round-trip time (mean and maximum, ms), received signal strength (dBm), negotiated transmit bitrate (Mbit/s), channel, and two operator judgements (video: good, choppy or lost; control: yes, degraded or no). The intended usable range is the largest distance at which both judgements hold and loss stays below 5%.

Table 5.7. Communication range and link degradation (E6), `network.csv`. MEASURED (directly logged by `netsweep.py`, not hand-transcribed); partial – 16 stations, line-of-sight, no obstacles. One 100-ping burst per station. Distances and the `video`/`control` judgements are as entered by the operator and were not independently verified. `channel`/`freq_mhz` were added to the logging tool after N1-N4 were run. N1-N6 were logged on 2026-10-22 and N7-N10, R1-R6 on 2026-10-23, in the row order shown (R1 was logged after N10 at 250 m).

label	dist_m	obst.	sent	recv	loss%	rtt avg	rtt max	sig start	sig end	tx Mbps	ch (MHz)	video	control
N1	5	none	100	100	0.0	3.4	18.0	-41	-41	72.2	-	-	-
N2	10	none	100	100	0.0	5.3	30.9	-39	-41	72.2	-	-	-
N3	15	none	100	100	0.0	5.7	41.7	-41	-41	65.0	-	-	-
N4	20	none	100	100	0.0	5.3	37.1	-40	-40	72.2	-	-	-
N5	25	none	100	100	0.0	4.8	31.7	-42	-42	72.2	3 (2422)	-	-
N6	50	none	100	100	0.0	3.5	25.4	-41	-40	72.2	3 (2422)	-	-
N7	100	none	100	100	0.0	3.9	47.1	-52	-50	72.2	10 (2457)	good	yes
N8	150	none	100	99	1.0	12.6	215.6	-55	-56	72.2	10 (2457)	good	yes
N9	200	none	100	100	0.0	42.9	103.3	-58	-53	72.2	10 (2457)	good	yes
N10	250	none	100	94	6.0	16.6	307.2	-60	-60	72.2	10 (2457)	good	yes
R1	5	none	100	98	2.0	11.9	93.0	-58	-57	72.2	10 (2457)	good	yes
R2	10	none	100	98	2.0	9.4	158.4	-55	-57	72.2	10 (2457)	good	yes
R3	15	none	100	100	0.0	5.5	122.8	-58	-60	72.2	10 (2457)	good	yes
R4	20	none	100	92	8.0	5.0	93.6	-57	-58	72.2	10 (2457)	good	yes
R5	25	none	100	90	10.0	6.3	144.7	-60	-55	72.2	10 (2457)	good	yes
R6	50	none	100	84	16.0	6.7	43.5	-55	-60	72.2	10 (2457)	good	yes

Table 5.7a. The same six distances measured in the two sessions (N1 to N6 on 2026-10-22, R1 to R6 on 2026-10-23), from Table 5.7. Loss in %, signal at the start of the burst in dBm, mean RTT in ms.

distance_m	loss N	loss R	signal N	signal R	rtt_avg N	rtt_avg R
5	0.0	2.0	-41	-58	3.4	11.9
10	0.0	2.0	-39	-55	5.3	9.4
15	0.0	0.0	-41	-58	5.7	5.5
20	0.0	8.0	-40	-57	5.3	5.0
25	0.0	10.0	-42	-60	4.8	6.3
50	0.0	16.0	-41	-55	3.5	6.7

What the sweep shows.

(i) *Distance did not degrade the link measurably over 5 to 250 m.* On 2026-10-23 the signal level at the start of each burst lay between -52 and -60 dBm at every distance from 5 to 250 m, and the negotiated transmit bitrate was 72.2 Mbit/s at all ten stations of that day. Signal level and distance are not monotonically related: the 5 m station R1 read -58 dBm and the 200 m station N9 read -58 dBm.

(ii) *Ping loss did not track distance.* Loss was 2.0, 2.0, 0.0, 8.0, 10.0 and 16.0% at 5, 10, 15, 20, 25 and 50 m (R1 to R6), and 0.0, 1.0, 0.0 and 6.0% at 100, 150, 200 and 250 m (N7 to N10). The highest loss in the data (16%) was at 50 m; the 100 to 200 m stations lost 1% or less. Loss over R3 to R6, the stations logged in succession, rose from 0 to 16% in about two and a half minutes (18:43 to 18:45 UTC) as the stations moved outward, which a time-dependent cause would also produce. Mean RTT ranged from 3.4 to 42.9 ms and maximum RTT from 18.0 to 307.2 ms, again without a monotonic relation to distance (the highest mean, 42.9 ms, was at 200 m, and the 250 m station averaged 16.6 ms).

(iii) *The two sessions disagree at the same distances.* At the six distances measured twice (Table 5.7a), the 2026-10-22 session showed 0% loss and about -41 dBm throughout, and the 2026-10-23 session showed 0 to 16% loss and -55 to -60 dBm. The 17 dB difference in signal at 5 m is larger than any change with distance within either session. The data do not identify why. Candidate explanations, none tested: a change in the access point's channel (N5 and N6 were logged on channel 3, 2422 MHz, and all rows of 2026-10-23 on channel 10, 2457 MHz), a change in the access point's position or the robot's antenna orientation between sessions (not recorded), co-channel interference on the shared 2.4 GHz band (no scan of nearby networks was run), or that some distances entered on one of the days were not the true distances. The operator confirmed the 5 m and 10 m positions of R1 and R2 when asked, but did not report whether the router was moved or restarted between sessions.

(iv) *The 5% loss condition and the operator's judgement disagree.* At all ten stations where judgements were recorded the operator judged video `good` and control `yes`, including N10 (6%), R4 (8%), R5 (10%) and R6 (16%), where loss exceeded the 5% limit of the metric definition. Read literally, the usable-range definition therefore has no single answer: the largest station passing all three conditions is N9 (200 m), but R4, R5 and R6 (20, 25 and 50 m) fail the loss condition, so the passing stations are not contiguous from the access point outward and 200 m is not a range boundary. By the operator's judgement alone the usable range is at least 250 m, with the real limit not reached. The 5% threshold is an a priori criterion of this paper, not a measured tolerance of the robot; whether the video and control actually tolerate 16% ping loss, or the judgements were made at coarse resolution, is not separable with these data. ICMP loss to the router is also not the loss rate of the control heartbeat: the heartbeat-loss sensitivity of Section 5.1b (Tables 5.5b, 5.5c) remains an assumption-driven SIMULATED result, and dead-man trips during the sweep were not logged.

What the sweep does not show. No station was reached at which video or control was judged choppy, lost, degraded or unusable, so the failure point, the usable-range upper bound, and the behaviour of the dead-man (Section 5.4, F7) at link loss are not measured here. No obstructed-link condition (1-wall, 2-walls) was run. Each station is one burst; the burst-to-burst variation at a fixed position was not measured, so the 0 to 16% loss spread cannot be separated into a distance effect, a time effect and sampling noise. Station timestamps (`iso_ts`) place N7 to R6 within 11 minutes (18:34:53 to 18:45:49 UTC), including the return from 250 m to 5 m between N10 and R1; this implies fast movement of the robot between stations, and the distances should be read as operator-entered values. The only other link-level observations in this paper are from the indoor laboratory, at a single location within 3 m of the access point: the mean signal level over each E8 condition lay between -40.7 and -49.0 dBm (`resources.csv`, MEASURED). These describe the test bench, not range.

Scope caveat. Every figure in this subsection is a property of the specific access point used for testing (SSID observed as `TOMAL`, 2.4 GHz), together with its transmit power, antenna, surroundings and neighbouring networks, and of the Pi's on-board radio, not an intrinsic bound on the robot. The access point's channel was not constant across the sweep: channel 3 / 2422 MHz at N5 and N6 (2026-10-22) and channel 10 / 2457 MHz on 2026-10-23, the channel stated in Section 3.2, which a live reading on 2026-10-23 matched. A different router or environment would very plausibly shift every number here in either direction; the paper should say so wherever a range or loss figure is cited.

[PENDING MEASUREMENT: E6 continuation.] Protocol: (a) continue the line-of-sight sweep outward from 250 m until the operator judges video or control choppy/degraded or worse, recording that distance as the usable range (or recording that the terrain ended first); (b) add obstacle conditions (1-wall, 2-walls) at representative distances; (c) repeat one fixed station several times in succession (for example three bursts at 50 m) to separate burst-to-burst variation from distance; (d) scan nearby networks (`iw dev wlan0 scan`) at the start of each session

to record channel occupancy, and record the router's position and channel at the start of each session; (e) measure distances with a tape or wheel rather than entering estimates; (f) decide, and state in advance, whether the usable-range criterion is the 5% loss threshold or the operator's judgement alone. The usable range will then be reported against the single-cell assumption of Section 3.2 and the under-1-Mbit/s steady-state budget of Table 1a.

5.4 Fault Injection and Recovery Time

Procedure (automated). The automated campaign (E7, Table 4.1) was run on 2026-10-07 with `tools/eval/faultinject.py run`, which sends SIGKILL to the target process and polls the health of the target about every 50 ms. Ten trials were run against each of P1 control (test F3), P2 media (F4) and the P3 watchdog (F5).

Metrics. `detection` is the time from the SIGKILL to the harness's first observation that the target no longer reports healthy; it bounds how soon the outage became externally visible, and it is not the time P3 took to notice the crash. `recovery` is the time from the SIGKILL to the health endpoint answering healthy with a new PID, so recovery means confirmed service, not merely a restarted process. Recovered is the proportion of trials meeting that condition.

Table 5.8. Fault detection and recovery, `tools/eval/results/faults.csv`, 10 trials per target. MEASURED.

Test	Target	Metric	Mean	95% CI	SD	Min	Max	Median	Recov.
F3	P1	detection (ms)	91.1	66.2 to 116.0	34.8	59.2	182.8	80.7	10/10
F3	P1	recovery (ms)	16026.7	15926.1 to 16127.3	140.6	15804.4	16254.8	15990.0	10/10
F4	P2	detection (ms)	123.2	90.2 to 156.2	46.1	59.8	175.0	147.1	10/10
F4	P2	recovery (ms)	14450.4	13762.9 to 15138.0	961.2	12966.8	15914.9	14758.4	10/10
F5	P3	detection (ms)	157.0	128.2 to 185.8	40.3	61.3	216.0	156.6	10/10
F5	P3	recovery (ms)	12829.0	12182.8 to 13475.2	903.3	10428.5	13841.4	12998.0	10/10

Table 5.8a. Recovery proportion with 95% Wilson score interval [**?**]. CALCULATED from Table 5.8.

Test	Recovered	95% interval for the recovery probability
F3 (P1)	10/10	72.2% to 100%
F4 (P2)	10/10	72.2% to 100%
F5 (P3)	10/10	72.2% to 100%
Pooled	30/30	88.6% to 100%

Analysis. All 30 trials recovered with a confirmed PID change; P1's event log independently shows ten serial-port re-opens during the F3 campaign, one per restart. Thirty successes bound the per-trial failure probability under these bench conditions below 11.4% at 95% confidence (Table 5.8a), which supports the process-supervision claim of Section 3.7.1 for SIGKILL faults on the bench but does not demonstrate long-run reliability. The P1 recovery mean of 16.03 s (95% CI 15.93 to 16.13 s) lies inside the 14.6 to 19.6 s window predicted by the recovery-time model of Section 3.7.2 (PREDICTED), and the small SD of 140.6 ms is consistent with a recovery path dominated by the fixed 10 s cooldown. P2 recovered in 14.45 s on average, faster than P1, consistent with P2 opening no serial link and performing no Arduino handshake; Section 3.7.2 gives no separate bound for P2, so this is a description, not a test. P3 recovered in 12.83 s on average; its restart is performed by systemd (`RestartSec=5` in `robot-watchdog.service`) followed by P3's own start-up, which is a different path from the cooldown-gated respawn P3 applies to its children. The detection figures are 59 to 216 ms in every trial, which shows that the outage is visible at the health endpoint within a fraction of a second; they do not measure P3's own crash detection, so the earlier inference that they "confirm the 1 s exit poll dominates detection" is withdrawn. During each recovery window the dead-man timer, measured in Table 5.2b and in F7 below, holds the motors stopped once commands cease, so the recovery interval is a loss-of-service window rather than a loss-of-safety window.

Procedure (manual watch). On 2026-10-21, `tools/eval/faultinject.py watch` polled both `/health` endpoints at 10 Hz and logged every state transition. The injection instants themselves were not logged by the harness; they are bounded below from P1's event log.

Table 5.9. Manually injected fault transitions (E7 manual), `fault_watch.csv`, 2026-10-21. MEASURED. `t_s` is seconds from the start of each watch.

label	t_s	field	from	to	held_s
F6-arduino-unplug	4.70	serial_connected	1	0	4.70
F7-wifi-drop	26.85	ws_clients	1	0	26.85
F7-wifi-drop	28.76	ws_clients	0	1	1.91

Table 5.9a. Manual faults timed from `var/log/p1_events.log`, 2026-10-21 (UTC). MEASURED.

Fault	Event	Time	Interval
F6	First P1 serial read failure	19:41:58.363	-
F6	Harness sees `serial_connected` 1 $\rightarrow$ 0	19:41:58.475	0.11 s after first failure
F7	Last controller heartbeat forwarded to the Arduino	19:52:48.530	-
F7	Firmware `PANIC DEADMAN` received by P1	19:52:50.550	2.02 s after last heartbeat
F7	WebSocket closed by P1 (`ws_clients` 1 $\rightarrow$ 0)	19:53:11.686	23.16 s after last heartbeat
F7	Operator dashboard reconnected (`ws_clients` 0 $\rightarrow$ 1)	19:53:13.592	1.91 s after close

F6, Arduino disconnect. The Arduino link was cut by unbinding its USB device from the kernel driver, which removes `/dev/ttyACM0` as a cable pull would. P1 logged its first serial read failure at 19:41:58.363 and the harness observed `serial_connected` turn false 0.11 s later, within about one poll period; because the unbind instant itself was not logged, the delay from the physical event to the first failure is not measured. P1 stayed up (PID unchanged), logged read and write failures at about 0.5 s intervals as described in Table 9, and continued to serve WebSocket clients and telemetry with `serial_connected` false, consistent with the process isolation of the three-process split. When the device was re-bound, the kernel assigned a new node (`/dev/ttyACM1`) and `serial_connected` stayed false until P1 was restarted and reopened the port, confirming the Table 9 recovery path ("port reopened when P1 restarts"): P1 does not reopen a removed device by itself, so recovery from an unplug requires a P1 restart.
F7, Wi-Fi drop. The `wlan0` interface was taken down for about 20 s. The robot's own log shows the fail-safe chain working: the controller's heartbeats, which P1 forwards to the Arduino as they arrive from the dashboard (every 1.0 s in this window rather than the configured 0.5 s, Table 5.9d), stopped after 19:52:48.530, and the firmware tripped its dead-man and reported `PANIC DEADMAN` 2.02 s later, matching the 2002 to 2022 ms firmware interval of Table 5.2b. This is the second independent observation of the dead-man under an injected fault rather than a scripted probe, and it corrects the earlier draft, which stated that the dead-man was not exercised because P1 kept heartbeating; P1 forwards the controller's heartbeats and does

not generate its own. The WebSocket itself was not closed during the outage: P1 closed it 23.16 s after the last heartbeat, and the dashboard reconnected 1.91 s after the close. The 23 s is consistent with the WebSocket keep-alive of Table 1a (ping every 20 s, 20 s timeout, `pi/p1_control/main.py`), which reclaims a dead socket only after up to 40 s; the earlier attribution to the operating system's TCP keep-alive was incorrect. Because the instant at which `wlan0` was restored was not logged, this record cannot establish whether the socket closed during the outage or shortly after the link returned. The safety-relevant conclusion does not depend on that: the robot stopped 2.02 s into the outage, more than 20 s before the control socket closed, so liveness was enforced by the firmware and not by the transport, exactly as Section 3.2 intends.

F9, GPS loss: not a valid measurement. An outdoor attempt on 2026-10-22 did not produce a valid controlled loss-and-recovery measurement, and no antenna-obstruction cycle was performed. The NEO-6M first reported a fix 1306.57 s after the watch started (`fault_watch.csv`), with 3 to 5 satellites; the receiver's power-on time and almanac state at that moment are unknown, so this is not a time-to-first-fix measurement in the datasheet's sense, against which the manufacturer specifies 27 s for a cold start with all satellites at -130 dBm (PUBLISHED REFERENCE, [**?**]). The `gps_fix` transitions that followed are mixed in origin (Table 5.9b): 14 of the 25 outdoor fix losses occurred within 3 s of a P1 `GPS_FAIL` port error, which coincided with diagnostic reads of the same serial port by a second program, while the other 11 outdoor losses and all 7 losses in the later indoor run (`F9-indoor`) had no nearby port error and are consistent with genuine loss of a weak solution. The earlier statement that all of them reflected port contention is therefore corrected. None of these was a controlled injection, so no F9 result is claimed.

Table 5.9b. `gps_fix` $1 \rightarrow 0$ transitions during the F9 attempt, matched to P1 `GPS_FAIL` events. CALCULATED from `fault_watch.csv` and `p1_events.log`.

Watch label	**Fix-loss transitions**	**Within 3 s of GPS_FAIL**	**Not near a GPS_FAIL**
F9-gps-loss (outdoor)	7	3	4
F9-gps-loss-run2 (outdoor)	18	11	7
F9-indoor	7	0	7

[PENDING MEASUREMENT: F9 GPS loss.] Protocol for the repeat: read GPS only through P1 (no second reader on `/dev/serial0`), establish a fix that has been stable for 5 min, cover the antenna for 30 to 60 s, and record the `gps_fix` true-to-false transition and the recovery time after uncovering; 5 cycles.

Unplanned heartbeat stalls (not an injected fault). The `F9-indoor` watch was still running at 14:01 UTC on 2026-10-22 and appended 16 further rows to `fault_watch.csv`, recording P1's health endpoint failing to answer twice, for 0.35 s and 0.78 s, with `serial_connected`, `gps_fix` and `ws_clients` unreadable for the same intervals. P1 was not restarted (the same process, PID 924, logged before and after). P1's event log shows that the heartbeats it forwards to the Arduino paused at the same moments (Table 5.9c): the first gap, 1.988 s, stayed inside the 2000 ms window and caused no trip, and the second, 2.080 s, exceeded it, and the firmware reported `PANIC DEADMAN` at the end of it and cleared the fault 13 ms after the next heartbeat. The cause of the stalls was not recorded, so they are reported for completeness under the validity rule of Section 4.2 and are not used as a fault-injection result; they are, however, a third, unscripted observation consistent with the dead-man tripping when, and only when, the 2000 ms window is exceeded, on a sample of three gaps.

Heartbeat interval actually observed. The dashboard is configured to send a heartbeat every 500 ms (`HEARTBEAT_PERIOD_MS`), and P1 forwards each one without rate limiting (`pi/p1_control/safety.py`), so the interval of `H` commands in `p1_events.log` is the interval the dashboard delivered. It was not constant (Table 5.9d). Over the 2026-10-21 probe session it was 0.50 s (median), but in the minutes before the F7 injection and before the unplanned stalls it was 1.00 s. The cause was not recorded; a dashboard tab whose timers are throttled by the browser is a candidate, but this was not tested. At a 1.00 s period a single lost heartbeat already leaves 2000 ms of silence (Table 5.5b), against two tolerated losses at the configured 500 ms rate, so the margin of Section 5.1b holds only while the dashboard keeps that rate. The `H` commands analysed here are the controller heartbeat (dashboard to P1 to Arduino). They are distinct from the Arduino's own compact heartbeat line (tag `HB`, every 1000 ms, Arduino to P1), which is an outbound status report and does not re-arm the dead-man (Section 3.2). The dead-man observations of this subsection are unaffected, since each depends only on the 2000 ms of silence.

Table 5.9d. Interval between heartbeats (`DEBUG_TX H`) written to the Arduino, from `var/log/p1_events.log.1`. MEASURED. Seconds.

Window (UTC)	Heartbeats	Median	10th to 90th percentile
2026-10-21 11:40 to 12:30 (E1 probe session)	5308	0.501	0.488 to 0.513
2026-10-21 19:51:00 to 19:52:49 (before F7)	84	0.997	0.977 to 1.027
2026-10-22 12:01:30 to 12:03:07 (before first gap)	192	0.502	0.375 to 0.618

Window (UTC)	Heartbeats	Median	10th to 90th percentile
2026-10-22 14:00:00 to 14:01:20 (before stalls)	80	1.000	0.903 to 1.035

Table 5.9c. Gaps of 1.9 s or more between controller heartbeats written to the Arduino, 2026-10-22 12:00 to 14:02 UTC, `p1_events.log`. MEASURED.

Gap start	Gap end	Gap (s)	Dead-man trip
12:03:04.453	12:03:06.355	1.902	None
14:01:06.488	14:01:08.476	1.988	None
14:01:15.516	14:01:17.596	2.080	PANIC DEADMAN at 14:01:17.616

Limitations. Only SIGKILL crashes were injected automatically; hangs (detected by the 35 s health-check path of Section 3.7.2), camera and microphone loss, and Robot Screen exit were not injected. The 10-trial samples bound, but do not establish, the failure probability. F6 and F7 are single injections each.

5.5 Resource Utilization on the Edge Device

Procedure. Resource use was logged with `tools/eval/reslog.py` (E8, Table 4.1) at 1 Hz: system CPU (as a percentage of the whole four-core processor), per-process CPU for P1, P2 and P3 (as a percentage of one core, so up to 100% per fully busy core), memory, per-process resident memory (RSS), SoC temperature, the firmware throttle word and its decoded flags, and interface statistics. Six labelled conditions exist: `idle` (2026-10-08, no operator connected), `idle-partial-10min` (2026-10-07, media process encoding for the Robot Screen only), `driving` (2026-10-21, robot driven continuously over the control WebSocket by a scripted client, no operator media stream, wheels clear of the ground, freshly rebooted Pi), `driving+media` (2026-10-21, as `driving` with an operator video stream active, after the Pi supply was upgraded), `driving+media-16kbps` (2026-10-21, as `driving+media` with the Opus bitrate set to 16 kbit/s instead of 32 kbit/s, Section 4.3), and `driving+media-throttled` (2026-10-21, the first `driving+media` capture, on the original USB power bank of Table 2).

Audit of the logger runs. Each row of `resources.csv` carries both a wall-clock timestamp and the logger's own elapsed time, so the start time of the logger that wrote it can be

recovered. Five conditions are single uninterrupted runs at a 1.000 s median spacing. The `driving+media` condition is not: its 299 rows come from two logger instances started 94 s apart (20:14:56 and 20:16:30 UTC) and writing under the same label, of which the first 196 s of one and the first 103 s of the other are absent from the file, so the rows span 196 s of wall-clock time, with 103 s sampled twice, rather than the 300 s stated in the earlier draft. The rows were not altered; their statistics are reported as logged, and the per-run figures are given in Table 5.10a to show that the splice does not change the result.

Table 5.10a. `driving+media` split by logger run. CALCULATED from `resources.csv`.

Logger start (UTC)	**Rows**	**Total CPU mean (%)**	**P2 CPU mean (% core)**	**P2 CPU SD**	**P2 RSS mean (MB)**
20:14:56	103	21.06	63.01	4.10	453.4
20:16:30	196	20.60	62.87	3.81	453.9
Pooled, as logged	299	20.76	62.92	3.91	453.7

Table 5.10. Resource utilisation, `tools/eval/results/resources.csv`. MEASURED. Total CPU is % of the four-core processor; P1 to P3 CPU is % of one core; memory in MB; temperature in deg C. N is the number of 1 Hz rows.

Condition	**Metric**	**N**	**Mean**	**SD**	**Max**	**p95**
idle	total CPU	1799	14.44	15.67	91.40	46.11
idle	P1 CPU	1799	2.89	1.09	8.00	5.00
idle	P2 CPU	1799	0.90	0.54	3.00	2.00
idle	P3 CPU	1799	0.12	0.33	2.00	1.00
idle	memory used	1799	1906.9	97.2	2134.7	2081.0
idle	SoC temp	1799	45.2	1.8	52.1	48.2
idle-partial-10min	total CPU	603	33.47	20.05	95.90	88.18
idle-partial-10min	P1 CPU	603	4.62	1.66	11.00	8.00
idle-partial-10min	P2 CPU	603	26.64	29.47	97.00	83.81
idle-partial-10min	P3 CPU	603	0.16	0.40	2.00	1.00
idle-partial-10min	memory used	603	2341.8	171.2	2520.1	2507.9
idle-partial-10min	SoC temp	603	47.2	1.7	52.6	51.1
driving	total CPU	309	20.70	8.66	61.20	40.98
driving	P1 CPU	309	3.74	0.96	8.00	5.00
driving	P2 CPU	309	1.12	1.37	8.00	2.00
driving	P3 CPU	309	0.12	0.34	2.00	1.00
driving	memory used	309	2429.3	31.5	2508.5	2478.2
driving	SoC temp	309	48.0	0.8	51.6	49.2

Condition	Metric	N	Mean	SD	Max	p95
driving+media	total CPU	299	20.76	3.89	45.40	28.04
driving+media	P1 CPU	299	3.81	0.64	6.00	5.00
driving+media	P2 CPU	299	62.92	3.91	92.00	71.00
driving+media	P3 CPU	299	0.10	0.30	1.00	1.00
driving+media	memory used	299	2271.7	26.7	2304.6	2303.3
driving+media	SoC temp	299	49.5	0.6	51.6	50.1

Table 5.10b. Mean per-process CPU with 95% batch-means interval (ten non-overlapping batches). CALCULATED from `resources.csv`. % of one core.

Condition	P1 CPU mean (95% CI)	P2 CPU mean (95% CI)	Total CPU mean (95% CI), % of processor
idle	2.89 (2.75 to 3.03)	0.90 (0.88 to 0.92)	14.44 (8.61 to 20.26)
idle-partial-10min	4.62 (3.93 to 5.32)	26.64 (7.48 to 46.06)	33.47 (23.11 to 44.01)
driving	3.74 (3.57 to 3.93)	1.12 (0.47 to 1.81)	20.70 (19.18 to 22.51)
driving+media	3.81 (3.72 to 3.92)	62.92 (62.22 to 63.67)	20.76 (20.00 to 21.47)
driving+media-16kbps	3.85 (3.69 to 4.02)	64.51 (63.30 to 65.82)	22.98 (20.69 to 25.54)
driving+media-throttled	8.79 (8.66 to 8.90)	118.10 (117.08 to 118.98)	68.46 (67.60 to 69.58)

Table 5.11. Per-process resident memory (RSS), same source. MEASURED. Mean values in MB.

Condition	P1 RSS	P2 RSS	P3 RSS
idle	44.9	97.4	32.3
idle-partial-10min	45.4	117.5	32.3
driving	44.7	98.2	32.2
driving+media	45.4	453.7	32.3
driving+media, 16 kbps	47.0	136.1	32.3
driving+media, original supply (throttled)	47.6	192.4	32.3

Throttling and supply. The throttle word decodes as defined by the platform vendor (bit 0 under-voltage now, bit 2 throttled now, bits 16 and 18 the corresponding has-occurred flags) [**?**].

No sample in the five conditions captured on adequate supply showed under-voltage, throttling or frequency capping: 0/1799 (idle), 0/603 (idle-partial), 0/309 (driving), 0/299 (driving+media) and 0/299 (driving+media, 16 kbps), and `under_voltage_ever` was clear throughout each. The first `driving+media` capture on the original USB power bank showed under-voltage and throttling on 276 of 299 samples (throttle word 0x50005) with the has-occurred flags set on all 299; its figures are listed separately in Tables 5.10b and 5.11 and are not used for the clean-envelope result. Under throttling, P2 used 118.1% of one core against 62.9% on the adequate supply, which is consistent with the same encoding work taking a larger share of a slowed processor, although the two captures also differ in time and P2 state and the comparison is descriptive only. The supply upgrade was therefore necessary for the combined driving and media load, and the Pi supply in Table 2 is a measured limitation of the original build.

Analysis. The comparisons below are between uncontrolled captures on different days, P1 and P2 uptimes and remote-desktop states, so they are descriptive and no causal effect is inferred from them. Three findings are robust across the batch-means intervals. First, the control process P1 stays below 4% of one core in every adequately supplied driving condition (upper 95% limit 4.02%), and its mean is essentially the same with and without the operator media stream (3.74% against 3.81%, overlapping intervals), so the media load does not appear on the control process, which is the isolation the three-process split of Section 3.1 is designed to provide (RQ4). Second, the operator media stream is carried by P2: 1.12% of one core while driving without a stream against 62.92% with one, so a single operator session costs about two thirds of one of the four cores, and this cost lands on a process that carries no drive or stop command. Third, the SoC temperature stayed at or below 52.6 deg C in every condition, well below the 80 deg C at which the vendor documents progressive throttling of the Arm cores [**?**], so none of these runs approached thermal throttling; the runs lasted 3 to 30 min indoors and do not establish behaviour over a full mission or at high ambient temperature. P1's increase from 2.89% idle to 3.74% while driving is an upper estimate of the motor-service cost, since the two captures are 13 days apart. Total memory use stayed between 1.9 and 2.5 GB of the 4 GB platform in every condition.

Bitrate ablation. Table 5.12 gives the `driving+media-16kbps` condition. P2 used 64.51% of one core (95% CI 63.30 to 65.82) at 16 kbit/s against 62.92% (62.22 to 63.67) at 32 kbit/s, so halving the Opus bitrate produced no detectable reduction in media CPU; the audio encoder's cost is evidently small beside the video encoding that dominates P2. The resident-memory difference in Table 5.11 (453.7 MB at 32 kbit/s, 136.1 MB at 16 kbit/s) cannot be attributed to the bitrate: P3's log shows that P2 was terminated and respawned at 20:23 UTC, between the end of the 32 kbit/s capture (20:21) and the start of the 16 kbit/s capture (20:26), so the 16 kbit/s run measured a freshly started P2 while the 32 kbit/s run measured one that had been serving sessions for longer. The earlier draft's explanation of the memory saving by encoder buffer sizes and its statement about the cost structure of the Opus encoder are withdrawn as unsupported. The setting used for this run is also not reproducible from version control: the committed code

fixes both Opus encoders at 32 kbit/s (`MIC_BITRATE` in `pi/p2_media/media.py` and `AUDIO_BITRATE` in `pi/p2_media/talkback.py`), so the 16 kbit/s value was applied by an uncommitted change that the label is the only record of. The effect of the bitrate on audio quality, which is the other side of this ablation in Section 4.3, was not measured.

Table 5.12. Resource utilisation, 16 kbit/s Opus ablation (E8), `resources.csv`. MEASURED. Same metrics and units as Table 5.10.

Condition	Metric	N	Mean	SD	Max	p95
driving+media, 16 kbps	total CPU	299	22.98	8.35	73.90	42.02
driving+media, 16 kbps	P1 CPU	299	3.85	0.94	8.00	5.00
driving+media, 16 kbps	P2 CPU	299	64.51	4.33	85.00	74.00
driving+media, 16 kbps	P3 CPU	299	0.11	0.31	1.00	1.00
driving+media, 16 kbps	memory used	299	1962.0	29.7	2061.9	2032.1
driving+media, 16 kbps	SoC temp	299	49.2	0.8	52.6	50.6

Limitations. The driving conditions were reported by the operator to have run with the wheels clear of the ground; the logs cannot confirm this (the robot has no encoder), and a later 2026-10-22 session run on the same stated premise took place with the robot on the floor, so the statement is unverified. If it holds, motor current and its effect on the supply are not represented. Conditions are single captures of 3 to 30 min. A remote-desktop session was active in the idle and driving conditions and adds an unmeasured load to total CPU and to the interface counters. The 16 kbit/s run is confounded by a P2 restart and its configuration is not in version control. The Table 4.1 plan of four conditions at 30 min each was met only for `idle`.

5.6 Sensor Accuracy and GPS Track Quality

5.6.1 GPS: start-up, static precision and track quality

These are different metrics and are reported separately. Start-up is the time from receiver start to first fix. Static precision is the scatter of fixes from a stationary receiver about their own centroid. Accuracy is the distance of fixes from a surveyed reference point; no surveyed reference was used in this work, so no accuracy figure is reported. Track quality concerns whether the 10 m distance filter of Section 3.10 suppresses drift and keeps genuine motion.

Procedure. Static precision (E9, Table 4.1) was measured with `tools/eval/gpsdrift.py`, which reads the fixed samples of a stationary robot from `var/log/telemetry.log`, takes their centroid as the reference, and computes the radial distance of each fix from it. The archived window of 2026-09-29 holds 6888 fixes over 129.1 min outdoors. The figures were recomputed independently from the raw telemetry for this section and agree with `gps_static.csv` to the reported precision.

Table 5.13. Static GPS precision (E9), `tools/eval/results/gps_static.csv`. MEASURED. Radial distance from the centroid, in metres.

Samples	Duration (min)	Mean	SD	RMS	CEP50	CEP95	Max	Mean sats
6888	129.1	14.14	10.45	17.58	8.9	30.46	120.4	4.8

Resolution of the logged coordinates. P1 holds each fix to six decimal places (about 0.1 m, `pi/p1_control/gps_reader.py`), and the 10 m track filter operates on those values, but the telemetry log that `gpsdrift.py` reads writes latitude and longitude with four decimal places (`pi/p1_control/telemetry_log.py`). At the test latitude of 23.45 deg N one step of 0.0001 deg is 11.12 m north-south by 10.20 m east-west (CALCULATED), and uniform rounding to that grid alone contributes an RMS radial error of about 4.4 m. The 6888 archived fixes occupy only 61 distinct grid positions. The precision figures of Table 5.13 are therefore limited by the logging resolution: their order of magnitude is meaningful, but the CEP50 of 8.9 m is smaller than one grid step and should not be quoted to sub-metre precision. The earlier statement that the CEP50 "lies just below the 10 m track filter" draws a comparison finer than the data can resolve and is withdrawn.

Table 5.13a. Static GPS windows recomputed from `var/log/telemetry.log` at four-decimal resolution. CALCULATED from MEASURED fixes.

Window	Fixes	Dur. (min)	Mean (m)	RMS (m)	CEP50	CEP95	Max	Sats	Within 10 m	Distinct pos.
2026-09-29 archive (E9)	6888	129.1	14.14	17.58	8.90	30.46	120.4	4.76	53.1%	61
2026-10-22 outdoor, during F9 attempt	1717	34.3	26.02	33.80	17.68	82.04	101.3	4.16	32.5%	44

The second window comes from the outdoor part of the F9 attempt (Section 5.4), during which the robot was not driven (`fw_state` READY on every sample) but a second program was intermittently reading the GPS serial port; it is not a valid static test and is listed for transparency only. The two centroids, from the same site 23 days apart, lie 4.27 m apart; because the robot was not placed on a surveyed mark either time, this separation is not an accuracy figure.

Startup. One observation exists: during the F9 attempt the receiver first reported a fix 1306.57 s after the watch started, with 3 to 5 satellites (Section 5.4). Receiver power-on time and almanac state were not recorded, so this cannot be compared with the datasheet time-to-first-fix of 27 s for a cold start, which applies with all satellites at -130 dBm and without aiding [**?**]. It indicates only that acquisition at this site and on this occasion was slow.

Interpretation. Both windows have low satellite counts (mean 4.76 and 4.16), and both show a heavy upper tail (CEP95 of 30 to 82 m, maxima above 100 m). The NEO-6M's nominal 2.5 m CEP (PUBLISHED REFERENCE) is specified for 24 h static operation with all satellites at

-130 dBm [**?**], conditions these short, low-satellite sessions did not provide, so the two are not directly comparable. What the data do support is the practical conclusion: with this receiver and antenna at this site, a stationary robot's reported position scatters by roughly 10 m in the median and by tens of metres in the tail. That is adequate for approach-level guidance of a rescue team but not for locating a victim within a building, which is the role Section 3.10 assigns to GPS beside the operator's camera view. Because about half of the logged fixes lie more than 10 m from the centroid, the 10 m track filter cannot be expected to suppress all drift of a stopped robot.

Track quality. The track that P1 builds with its 10 m filter, on its six-decimal coordinates, was read from P1's memory through its read-only `GET/api/gps-track` endpoint on 2026-10-22 at 14:08 UTC and preserved as `tools/eval/results/gps_track_live_2026-10-22.geojson` (MEASURED). P1 had run without restart since 11:39:41 UTC, so the track covers every fix of that day: 2431 fixed samples from 12:23:46 to 13:14:04 UTC, during which the robot was never driven (`fw_state` READY on every sample) but was moved once by hand from the outdoor area to the indoor laboratory over a distance that was not recorded. The track holds 130 points and 1609 m of path (Table 5.13b). A single building-scale relocation cannot account for more than a small part of that path, so most of it is drift that passed the filter: under these low-satellite conditions (3 to 5 satellites) the 10 m filter did not keep a stationary robot's track clean. Because the track points carry no timestamps, the drift cannot be separated exactly from the relocation, and this is one session; the result nevertheless contradicts the design claim that drift around a stopped robot adds no track points, and Section 3.10 is corrected accordingly. This bears only on the track the operator sees for routing and for reporting the victim's position after the mission, not on live navigation: the robot carries no other localisation input and performs no mapping (Section 3.10 pairs GPS with the operator's own camera view for fine positioning), so a 1609 m drift path recorded while the robot sat still means an operator or rescue team reading that track back after a mission could be shown a route the robot never drove, over a distance an order of magnitude larger than the building it was in; the live dashboard position at any instant is still the single most recent fix, so this is a defect in the stored breadcrumb trail specifically, not in the instantaneous position report.

Table 5.13b. GPS track built by P1's 10 m filter, 2026-10-22, robot not driven. MEASURED (track) and CALCULATED (distances).

Points	**Path (m)**	**Step min**	**Step median**	**Step max**	**Median dist. from centroid (m)**	**Max (m)**
130	1609	10.0	10.7	51.9	26.2	91.6

[PENDING MEASUREMENT: E9 at full resolution and controlled GPS track quality.] Protocol: log fixes at six decimal places (or the raw NMEA), place the robot on a surveyed or survey-grade

reference point for at least 30 min with a stable fix and without moving it, report precision and accuracy separately, export the GPS track (with point timestamps) over the same period to count spurious points, then drive a measured course and compare the track length with the tape-measured course length.

5.6.2 Ultrasonic range (E10): [VERIFICATION REQUIRED]

Ultrasonic range accuracy was entered with `tools/eval/record.py ultrasonic`: a flat target at each of five tape-measured distances (20, 75, 150, 284 and 389 cm), the telemetry range reading recorded ten times at each, the error being reading minus true distance.

Table 5.14. Ultrasonic range error by true distance (E10), `tools/eval/results/ultrasonic.csv`. [VERIFICATION REQUIRED: contradicted by the telemetry log; not used to support any claim.] All values in cm; N = 10 at each distance.

True distance (cm)	N	Mean error	SD	Min error	Max error
20	10	0.0	0.0	0.0	0.0
75	10	0.0	0.0	0.0	0.0
150	10	0.0	0.0	0.0	0.0
284	10	0.0	0.0	0.0	0.0
389	10	0.0	0.0	0.0	0.0
All	**50**	**0.0**	**0.0**	**0.0**	**0.0**

Verification. The protocol records the telemetry range value, and the robot's telemetry log stores that same value about once per second. Between 18:30 and 19:05 UTC on 2026-10-21, the period spanned by the entries, the logged range was predominantly 30 to 31 cm, then 10 to 11 cm, then 18 to 20 cm, and no 75, 150, 284 or 389 cm reading occurs in that window; readings of 284 or 389 cm do not occur at any time in the retained telemetry (2026-10-18 to 2026-10-22). A 150 cm reading does occur twice elsewhere in that period (2026-10-21T11:46:33 and 2026-10-22T11:18:50 UTC), but in each case as a single sample inside a rapidly changing sequence (for example 167, 170, 150, 131, 111 cm one second apart) unrelated to this session, not as a sustained reading at a fixed target, so neither occurrence corroborates the ten 150 cm trials entered here. During the 20 cm entries the telemetry mostly read 19 cm (301 samples) rather than the exactly 20 cm entered (4 samples). Forty of the fifty entries therefore have no counterpart in the robot's record for the session; of those, the twenty 284 and 389 cm entries have no match anywhere in the retained record, the ten 150 cm entries are matched only

by the two brief, unrelated samples above, and the remaining ten (20 cm) are not supported to the stated zero error. The firmware's own range cap (`ULTRASONIC_MAX_CM = 400` in `arduino/config.h`) is not the explanation: 389 cm is below that cap and within the 25 ms `pulseIn` timeout's roughly 4.2 m reach (`ULTRASONIC_TIMEOUT_US`), so the sensor was capable of reporting a value anywhere up to 389 cm if one had been in front of it; the telemetry simply shows it was not.

A second, independent irregularity in the entered data itself, not only in its agreement with the logs, points the same way: `ultrasonic.csv`'s `reading_cm` column (documented by `tools/eval/record.py` as "telemetry value") is bit-for-bit identical to `true_cm` (the tape-measured distance) in all 50 rows, at all five distances from 20 to 389 cm, giving a stated error of exactly 0.0 cm on every trial with zero variance. `record.py` has no default or auto-fill between these two fields: both `true_cm` and `reading_cm` are listed as separately `required` inputs (`SHEETS["ultrasonic"]` in `tools/eval/record.py`), so this is not a software artifact of the entry tool; the two columns were typed to the same value by hand 50 times. No accuracy specification for this HC-SR04 unit is cited in this paper, but fifty-for-fifty exact agreement with the tape measurement at every one of five distances from 20 to 389 cm, with zero variance at each, is not a plausible outcome of reading an actual ultrasonic echo, which is subject to pulseIn timing quantisation, target angle and surface, and ordinary measurement noise; the pattern is the one expected if the tape-measured distance was entered into both columns. The table is retained as entered, but the earlier conclusion that the HC-SR04 returned the exact distance at every range up to 389 cm is withdrawn, and no row of Table 5.14, not only the forty contradicted by the telemetry log, is treated as a verified sensor reading.

[PENDING MEASUREMENT: E10 repeat.] Protocol: run `record.py ultrasonic` with the reading taken automatically from telemetry at the moment of entry (or log `range_cm` from `telemetry.log` over a marked interval), ten distances across 2 to 400 cm, a flat perpendicular target, 10 readings each; report mean error with 95% CI and SD per distance.

5.6.3 Temperature and humidity (E11)

Procedure. On 2026-10-21 the DHT11 reading was compared with a co-located reference thermometer-hygrometer at indoor ambient conditions; five trials were entered at one set-point. The error is sensor minus reference. The reference instrument's model and its own accuracy were not recorded.

Table 5.15. Temperature and humidity error (E11, DHT11 sensor), `tools/eval/results/dht11_environmental.csv`. MEASURED (transcribed; sensor values corroborated by `telemetry.log`); [VERIFICATION REQUIRED] for the `error_c` column (see below). Single ambient set-point, 5 trials.

trial	ref_c	sensor_c	ref_rh	sensor_rh	error_c	error_rh
1	31	31	77	77	0	0
2	31	31	77	77	0	0
3	31	31	77	77	0	0
4	31	31	77	78	0	+1
5	31	31	77	78	0	+1
All					**Mean 0.0**	**Mean +0.4**

Analysis. The robot's telemetry log over the same period (19:10 to 19:22 UTC) shows the DHT11 at 31 deg C throughout and alternating between 77% and 78% RH, which matches the transcribed sensor values. The temperature agreed with the reference on all five trials and the humidity differed by 0 or +1% RH (mean +0.4% RH, SD 0.55), at the one-unit resolution of both instruments. With one set-point, five readings and an uncharacterised reference, this shows only that the DHT11 agreed with the reference to within one unit at about 31 deg C and 77% RH; it does not characterise the sensor across its operating range, and the five readings, taken within minutes, are not independent. The tolerance usually quoted for the DHT11, 0 to 50 deg C at +/-2 deg C and 20 to 80% RH at 5% accuracy with a sampling rate of at most 1 Hz [**?**], matches the figures given here, but the source is a distributor-hosted spec sheet rather than a page reachable on the manufacturer's (Aosong) own site, and it is not relied on in the analysis above, which uses only the paired reference reading. The DHT11 feeds an advisory dashboard card and triggers no motor action (Section 3.5.3, Table 7).

Verification caveat on the error figures. `error_c` is exactly 0.0 deg C on all five trials, with no variance. The robot's telemetry log corroborates the sensor *values* that produced that figure (31 deg C and 77-78% RH), which is the agreement checked above, but a zero-error, zero-variance result on every trial of a hand-entered reference-minus-sensor difference is the same reading-equals-reference pattern independently found in `ultrasonic.csv` (Section 5.6.2), where the entered reading and the entered reference were shown to be typed to the same value rather than read independently 50 times. No record of the reference thermometer-hygrometer's readings independent of the entered `ref_c`/`ref_rh` values exists for this session, so the same explanation cannot be ruled out here. Telemetry corroboration of the sensor *values* is not evidence about the *error* column, since the error column compares the

sensor to a hand-entered reference that telemetry cannot check. [VERIFICATION REQUIRED: the `error_c` figure, and by the same reasoning the `error_rh` figure, not the corroborated sensor values.] The +/-0 deg C figure in this table must not be cited as measured sensor accuracy; it is retained because the sensor values themselves are corroborated by telemetry, but a defensible accuracy claim requires a repeat in which the reference instrument is read and entered independently per trial, the way `gas.csv` already was.
[PENDING MEASUREMENT: E11 additional set-points.] Protocol: repeat at four further set-points spanning the DHT11 operating range (for example near 20, 25, 35 and 40 deg C, and at low and high humidity) against a reference of stated accuracy, 5 readings per set-point, as planned in Table 4.1.

5.6.4 Gas sensor response (E12)

Procedure. On 2026-10-21 the MQ-136's raw ADC value (`gas_ppm`, which carries the 0 to 1023 count, not a calibrated concentration; Section 3.3, Table 2) was monitored while butane was released from an unlit lighter held about 5 cm from the sensor, five puffs in one session. Per trial, the baseline is the reading before the puff, the peak the maximum after it, and the trip time the interval from release to the first reading at or above the 450-count warn threshold. The ADC value updates on the firmware's slow-sensor cycle of about 2 s, so trip times are resolved to about +/-2 s. The warn and critical thresholds are 450 and 600 counts (Section 3.5.3, Table 7).

Table 5.16. Gas sensor response (E12), `tools/eval/results/gas.csv`. MEASURED (transcribed; corroborated by `telemetry.log`). Butane lighter at about 5 cm, 5 trials, 2026-10-21.

trial	source	baseline_adc	peak_adc	rise_adc	trip_s	card_colour
1	butane-lighter-5cm	216	935	719	1	red
2	butane-lighter-5cm	272	930	658	2	red
3	butane-lighter-5cm	180	864	684	2	red
4	butane-lighter-5cm	117	733	616	4	red
5	butane-lighter-5cm	101	910	809	2	red

Analysis. The telemetry log corroborates the session: the reading was steady at 50 to 55 counts from 19:25 to 19:30 UTC and then showed 12 separate rises to a local peak of at least 250 counts between 19:30 and 19:36 (peaks 280 to 935). The ten tabulated baseline and peak values all occur in the telemetry, in the order of Table 5.16, during 19:31:58 to 19:32:59, including the intermediate reading of 312 noted for trial 4; the five tabulated trials are therefore one consecutive run of puffs from a longer session, and the other exposures were not tabulated. The tabulated baselines (101 to 272) are the elevated minima between puffs, not the clean-air level of about 53 counts. On every trial the peak exceeded the 600-count critical threshold (range

733 to 935, mean 874) and the card turned red; the rise was 616 to 809 counts (mean 697.2, SD 72.9), and the warn threshold was crossed 1 to 4 s after release (mean 2.2 s, SD 1.1), at the resolution of the 2 s sensor cycle. The sensor is specified for hydrogen sulphide but responds to butane, and because the firmware reports raw counts, the result shows only that the sensor and the dashboard alert respond to a nearby gas source within a few seconds; it says nothing about the concentration of any specific gas or about the detection limit. The five puffs within one session are not independent trials. The reading drives an advisory card and never a motor action (Section 3.5.3, Table 7).

5.7 Field Trial: End-to-End Rescue Scenario

This subsection reports the end-to-end rescue runs (E13, Table 4.1) over an indoor corridor course with a role-played victim, under the human-subject protocol of Section 4.1. No per-run record of a field trial exists in the repository or on the robot: there is no `field.csv`, and `tools/eval/results/summary.md` lists E13 as not yet measured. The experiment is therefore reported as pending.

[PENDING MEASUREMENT: E13 field trial.] Protocol: drive the robot through a measured corridor course to a concealed role-played victim, establish two-way interaction, and record each run with `tools/eval/record.py field`, which writes one row per run including time to victim, total time, collisions, unplanned stops, and whether the operator saw and heard the victim and the victim heard the operator; 5 to 10 runs, with the operator, course layout and any aborted run recorded. The robot's own logs (`p1_events.log`, `telemetry.log`, `p2_events.log`) must be retained for each run so that every row can be cross-checked.

Success criteria. A run passes when (i) the robot reaches the victim position, (ii) two-way communication succeeds in both directions (operator sees and hears the victim; victim hears the operator), and (iii) no collision forces an unplanned stop. Each criterion is also reported on its own.

Metrics. Mission success proportion with a 95% Wilson interval [**?**]; time to victim and total time (s) with mean, 95% CI, median and range; collisions per run; unplanned stops per run; two-way communication success proportion with a 95% Wilson interval.

Table 5.17. End-to-end rescue runs (E13), `field.csv`. [PENDING MEASUREMENT]

run	course	time_to_victim_s	total_time_s	collisions	unplanned_stops	saw	heard	victim_heard	result

Table 5.17a. Field-trial summary (E13). [PENDING MEASUREMENT]

Metric	Value	95% interval
Runs		-
Mission success		
Time to victim, mean (s)		
Total time, mean (s)		
Collisions per run, mean		
Two-way communication success		

[PENDING MEASUREMENT: E13 analysis to be filled when field.csv data is collected; the success rate and two-way-communication success will be assessed against the two-way victim-interaction contribution (C3, RQ4), with the platforms of Table 2.1 (Section 2.6), none of which reports two-way audio and video with the victim on single-board hardware, as the qualitative baseline.]

5.8 Operator Usability Evaluation

This subsection reports the operator usability study (E14, Table 4.1). No participant has been tested and no questionnaire response exists; the experiment is entirely pending.

Protocol. Eight to twelve volunteer operators, recruited and consented under the protocol of Section 4.1, each receive the same scripted briefing, complete one practice drive, and then perform the standard course once with the dashboard in Chromium on the operator laptop. Task time, task completion and error count (collisions and wrong turns) are recorded by the experimenter, after which the participant completes the ten-item System Usability Scale [**?**] on a 1 to 5 agreement scale. Responses are logged with `tools/eval/record.py sus`, one row per participant, identified by participant number only.

Reporting requirements. The paper will report the number of participants recruited, completing and excluded (with the reason for each exclusion); each participant's prior experience with remote-controlled vehicles or games on a stated ordinal scale; the order and timing of sessions; and any deviation from the script. No personally identifiable information is stored.

SUS scoring. Each odd-numbered item contributes its response minus 1, and each even-numbered item contributes 5 minus its response; the ten contributions are summed and multiplied by 2.5 to give a score from 0 to 100 [**?**], as implemented by `sus_score` in `tools/eval/record.py`. A SUS score is not a percentage. The study mean will be

reported with SD, 95% CI and median, and positioned against the empirical distribution of SUS scores reported by Bangor et al. [**?**]; with 8 to 12 participants the interval will be wide, and the result will be framed as a formative usability check rather than a population estimate.

Table 5.18. Operator usability (E14), `sus.csv`. [PENDING MEASUREMENT]

p	q1	q2	q3	q4	q5	q6	q7	q8	q9	q10	time_s	compl.	err	sus

Table 5.18a. Usability summary (E14). [PENDING MEASUREMENT]

Metric	N	Mean	SD	95% CI	Median
SUS score (0 to 100)					
Task time (s)					
Errors per participant					
Task completion				(Wilson)	-

[PENDING MEASUREMENT: E14 analysis to be filled when sus.csv data is collected; the mean SUS score and task-completion rate will be reported, and the study's size and scope framed against the larger operator study deferred to Future Work (Section 7).]

5.9 Cost Analysis and Comparison with Existing Systems

This subsection compares the build cost of the robot with the cost reported by the platforms of the Section 2.6 comparison (E15, Table 4.1). The component list is Table 2 of Section 3.3; the per-line bill of materials is Appendix A (Table 5.19).

Evidence class. The prices in Table 5.19 are the actual amounts the authors paid, taken from their itemised procurement record (a shared spreadsheet, most lines carrying a linked image of the purchase as buying proof). They were paid in Bangladeshi Taka (BDT) and are converted to US dollars at an indicative rate of 1 USD = 120 BDT (October 2026): the BDT figures are receipt-verified, while the US-dollar total carries the uncertainty of that single exchange rate, which the authors should confirm with the rate and date actually used. The record is not a complete per-component bill of materials: several Table 2 items are not listed separately – the USB webcam, the speaker and the Pi's 50000 mAh power bank do not appear, and the GPS receiver, the two servos, the ultrasonic sensor and the DHT11 are not itemised but fall within one grouped sensors-and-circuit-board purchase – so the total below is a receipt-verified figure for the parts that are recorded, and the few unrecorded commodity parts would add a small

amount on top. Reusable tools (a soldering kit and iron, a nickel-strip welding consumable and a woodworking service), labour, report printing and a software subscription appear in the same sheet and are excluded here as not part of the robot.

Table 5.19. Appendix A bill of materials, by subsystem, from the authors' itemised procurement receipts. Actual prices paid in BDT; USD converted at an indicative 1 USD = 120 BDT (October 2026). Component parts are those of Table 2 of Section 3.3; a few Table 2 parts are not separately itemised (see below).

Subsystem group	**Cost (BDT)**	**Cost (USD)**
Edge computer, storage and display (Pi 4, 7-inch LCD, micro-HDMI cable, 64 GB microSD, case with fan)	20,861	173.84
Microcontroller and motor drive (Arduino UNO, 4 geared motors, 2 x BTS7960)	6,802	56.68
Sensors (MQ-136, plus one grouped sensors-and-circuit-board purchase)	4,007	33.39
Power subsystem (4S3P 18650 pack and cells, cell holder, 2 x buck, fuse, 14 AWG wire, capacitors, switch)	3,946	32.88
Wiring, connectors and mounting (jumper wires, breadboard, resistor, barrel jack, heat-shrink, foam tape, ties, clips)	782	6.52
Chassis and body	700	5.83
Shipping (within Bangladesh)	60	0.50
Receipt-verified total	**37,158**	**approx. 310**

Missing and changed items. The battery is now specified: a 4S3P pack of twelve 18650 Li-ion cells of 1500 mAh each (4.5 Ah nominal at 14.8 V) with a 40 A BMS (Table 2); its cells, holder and protection are in the power-subsystem line above. Three parts of Table 2 are absent from the receipt record and are not priced here: the USB webcam, the speaker and the Pi's 50000 mAh power bank. The first two are low-cost commodity items; the power bank is the supply that Section 5.5 shows under-volted under the combined driving and media load and was replaced on 2026-10-21, with the replacement likewise not recorded. The GPS receiver, the two servos, the ultrasonic sensor and the DHT11 are not itemised and lie within the grouped sensors purchase,

so they cannot be priced to a line. The operator laptop and the Wi-Fi access point are excluded, as before.

Table 5.20. Build cost comparison with literature platforms (E15). This work: receipt-backed (BDT, USD at an indicative rate); other rows: PUBLISHED REFERENCE. Figures are taken from the cited papers as published and not re-costed; "not reported" means the cited paper states no numeric total.

Platform	**Reference**	**Reported total cost**	**Note**
This work	-	approx. USD 310 (37,158 BDT at 1 USD = 120 BDT; receipt-verified for the itemised parts, Table 5.19)	Low-cost single-board teleoperated rescue robot
TXRob	[**?**]	Not reported (described as low-cost)	Paper states the low-cost design goal but gives no numeric BOM total or price
Paripreksya 2.0	[**?**]	Not reported in the full text	Competition-grade tracked robot with 4-DoF manipulator; the word cost does not occur in the full text read on 2026-10-23
WTC response robots	[**?**]	Not reported	Field study of commercially procured robots; no unit-cost breakdown given
Rescue reconnaissance platforms	[**?**]	Not reported	Requirements-and-challenges study; no single platform cost stated
USAR survey platforms	[**?**]	Not reported	Control-perspective survey; no common cost figure across the surveyed systems

Analysis. From the authors' receipts the itemised parts sum to 37,158 BDT, approximately USD 310 at an indicative 1 USD = 120 BDT, dominated by the Pi with its display and storage (about USD 174) and the motor drive (about USD 57) rather than by any specialised part; the few unrecorded commodity items (webcam, speaker, power bank) would add a small amount.

None of the cited comparison platforms reports a numeric build cost: TXRob [**?**] is explicitly designed as low-cost but states no figure, Paripreksya 2.0 [**?**] gives no cost in its full text, and the remaining references are a field study [**?**], a requirements study [**?**] and a survey [**?**] rather than single-platform build reports. The comparison here and in Section 6.1 is therefore qualitative plus the figures the papers state, as framed in Section 4.3: a cost ratio between this robot and those platforms cannot be computed, and the earlier statement that the platform is "an order of magnitude cheaper" than them is withdrawn, since it rested on no reported figure. The affordability contribution (C4) is now supported by receipts: the robot's recorded parts are commodity single-board, microcontroller and hobby-grade components totalling about USD 310, in place of the earlier estimate, with the currency (BDT) stated and the US-dollar figure resting on the single indicative exchange rate above.
[E15 cost, receipt-backed. The per-line estimate of the earlier draft has been replaced by the authors' itemised receipts (Table 5.19). Residual items before the figure is final: confirm the exchange rate and date actually used for the BDT-to-USD conversion; add the USB webcam, the speaker and the Pi power bank (and any replacement supply), which the receipt record omits; and, if a per-component rather than per-subsystem breakdown is wanted, split the grouped sensors-and-circuit-board purchase into its parts.]

5.10 Summary of Evidence

The strongest findings supported by the existing data are: (i) the firmware removes motor drive within 2042 ms of the last command in all 30 logged dead-man trials (mean 2022.5 ms on P1's clock), below the approximately 2060 ms analytical bound, and the dead-man also tripped 2.02 s into an injected Wi-Fi outage, more than 20 s before the control socket closed, and on an unplanned 2.080 s heartbeat gap while a 1.988 s gap did not trip it (Tables 5.2b, 5.9a and 5.9c); (ii) the watchdog restored service with a new PID in 30 of 30 SIGKILL trials, with the P1 recovery time of 16.03 s (95% CI 15.93 to 16.13 s) inside the 14.6 to 19.6 s model window (Table 5.8); (iii) on an adequate supply the control process stays below about 4% of one core while one operator media session costs about 63% of one core on the media process, with no throttling and SoC temperatures at or below 52.6 deg C (Tables 5.10 and 5.10b); and (iv) the original power bank could not sustain driving with media without under-voltage. The thermal and gas sensor *values* are corroborated by the robot's own logs but are single-session, single-condition checks; the thermal *error* figures of Table 5.15 carry the same [VERIFICATION REQUIRED] caveat as the ultrasonic table, for the same reading-equals-reference reason (Sections 5.6.2, 5.6.3). Software verification of the dead-man, command-validation and command-framing logic (146 unit tests, the 121-test suite plus 25 supplementary tests, Table 5.5a) and a SIMULATED heartbeat-loss sensitivity analysis (Tables 5.5b, 5.5c) add software-logic and analytical evidence to the dead-man argument; neither class of evidence substitutes for the pending physical measurements of E2, E4, E13 and E14, nor for completing E6. A further native test against

the real, compiled Arduino source (Table 5.5d) found that two properties the mock-based tests reported as "documented" firmware behaviour did not, at the time, hold on the firmware itself (the dead-man trips at, not after, exactly 2000 ms of silence, and pan/tilt/status commands do re-arm it); `pi/common/mock_hardware.py` and `pi/common/protocol.py` have since been corrected on both points (Section 5.1a addendum), so the dead-man timeout is now understood, and modelled, as running from the last valid command of any kind, not specifically the last drive command. The native test also confirmed the motor enable-line and PWM pins reach a safe (disabled, zero) state on trip and that recovery after a timeout is automatic within one 10 ms safety-task pass once any valid command resumes. This changes which claims the dead-man argument may rely on, not the argument's conclusion: the firmware still removes drive within the measured 2042 ms bound in every logged trial regardless of which boundary direction or arming rule is in force. The control-plane latencies are robot-internal upper bounds. GPS precision is resolution-limited by the logging format, and the one available track shows the 10 m filter admitting drift from a robot that was not driven (Table 5.13b). The emergency-stop distances and the ultrasonic accuracy table are not supported by the robot's logs and require verification or repetition. Streaming (E5), true control latency (E2), physical stop time (E4), the field trial (E13) and the usability study (E14) have no data, while the build cost is now receipt-backed at about USD 310 (37,158 BDT; Section 5.9). The range sweep (E6) is partial: 16 line-of-sight stations to 250 m found no failure point (usable range at least 250 m by operator judgement), ping loss of 0 to 16% that did not track distance, a 17 dB signal difference between two sessions at the same distances that is unexplained, and no obstacle condition.